Chapter 4: Predicting Chemical Outcomes – Supervised Learning for Classification and Regression

Chem 361

Xuhui Huang

Supervised Learning

ML: **Input Features** → **Training (Learning)** → **Generation (Prediction)**

Input features: $\mathcal{X}$

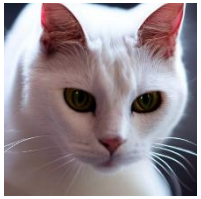
Target: $\mathcal{Y}$

Learn the mapping f to predict $\mathcal{Y}$ based on $\mathcal{X}$

$\hat{\mathcal{Y}} = f(\mathcal{X})$

Labels:

"black cat" "white cat" "dog"



Predict → "dog"

There are various classes of models you can select to approximate f!

For example:

- Linear regression models — **Parametric methods**
- K-Nearest Neighbors
- Decision Trees and Random Forest

Non-parametric methods

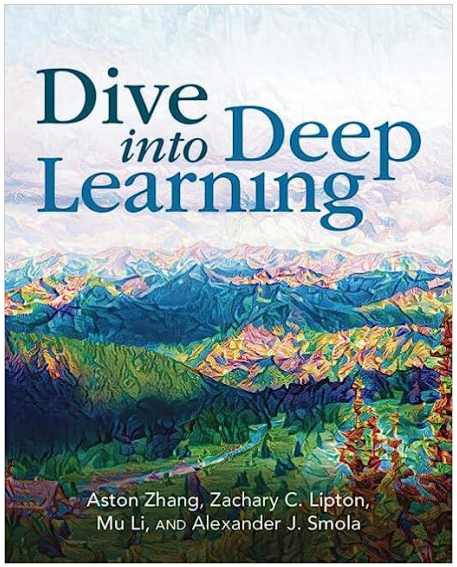
And different types of neural networks! - **Chapter 5**

© Xuhui Huang

Outline

- **Parametric Methods**
  - **Linear Regression**
  - Logistic Regression
  - Avoiding Overfitting
- Non-parametric Methods
  - K-Nearest Neighbors (KNN)
  - Decision Trees
  - Random Forest

Reading material:



Chapter 3

https://d2l.ai/

© Xuhui Huang

Linear Regression

Assume that the expected value of the target (e.g., LogS) can be expressed as a weighted sum of the features (e.g., molecular weight (mw)):

$$LogS = \theta_{mw} * mw + \theta_0$$

General expression of linear regression

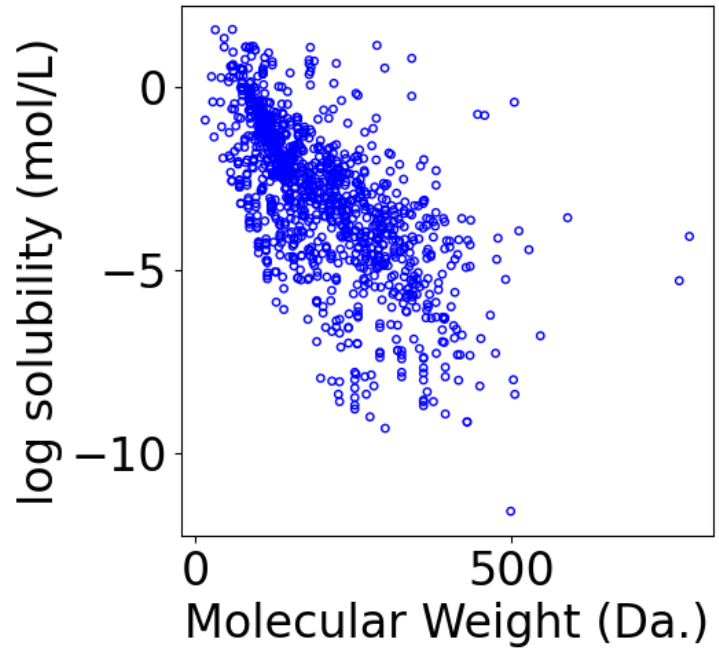
$$f_\theta(x) = \sum_{i=0}^{d} \theta_i x_i = \theta^T x$$

When $d = 1$, $f_\theta(x) = \theta_1 x + \theta_0$

Task: Find parameter θ that minimizes mean squared error (MSE) – loss function:

$$\ell(f_\theta) = \frac{1}{n}\sum_{j}^{n}\left(f_\theta\left(x^{(j)}\right) - y^{(j)}\right)^2$$

J. Chem. Inf. Comput. Sci. 44, 1000, (2004)



Delaney dataset with n=1128 compounds:

$$((x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)}))$$

© Xuhui Huang

Matrix Notation

Features Parameters Predictions

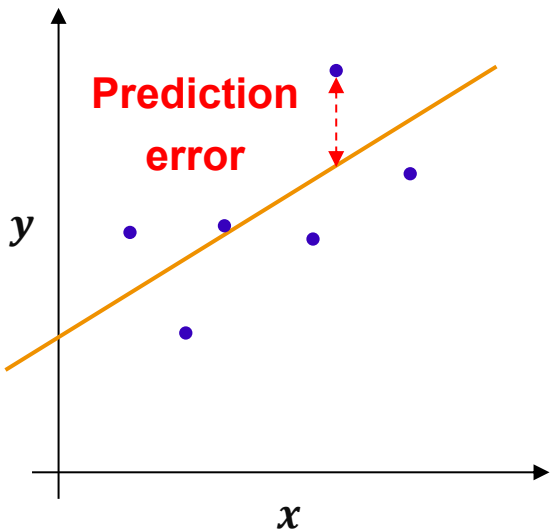
$$\begin{bmatrix} 1 & x^{(1)} \\ 1 & x^{(2)} \\ 1 & \dots \\ 1 & x^{(n)} \end{bmatrix} \times \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} \theta_0 + \theta_1 x^{(1)} \\ \theta_0 + \theta_1 x^{(2)} \\ \dots \\ \theta_0 + \theta_1 x^{(n)} \end{bmatrix}$$

X θ f_θ

Prediction Error

$$\begin{bmatrix} \theta_0 + \theta_1 x^{(1)} \\ \theta_0 + \theta_1 x^{(2)} \\ \dots \\ \theta_0 + \theta_1 x^{(n)} \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(n)} \end{bmatrix} = \begin{bmatrix} f_\theta\left(x^{(1)}\right) - y^{(j1)} \\ f_\theta\left(x^{(2)}\right) - y^{(2)} \\ \dots \\ f_\theta\left(x^{(n)}\right) - y^{(n)} \end{bmatrix}$$

f_θ y $X\theta - y$



Loss function – MSE:

$$\ell(f_\theta) = \frac{1}{n}\sum_j^n \left(f_\theta\left(x^{(j)}\right) - y^{(j)}\right)^2 = \frac{1}{n}(X\theta - y)^T(X\theta - y) = \frac{1}{n}||X\theta - y||_2^2$$

© Xuhui Huang

Analytical Solution

We can find the optimal parameters (θ) analytically for linear regression:

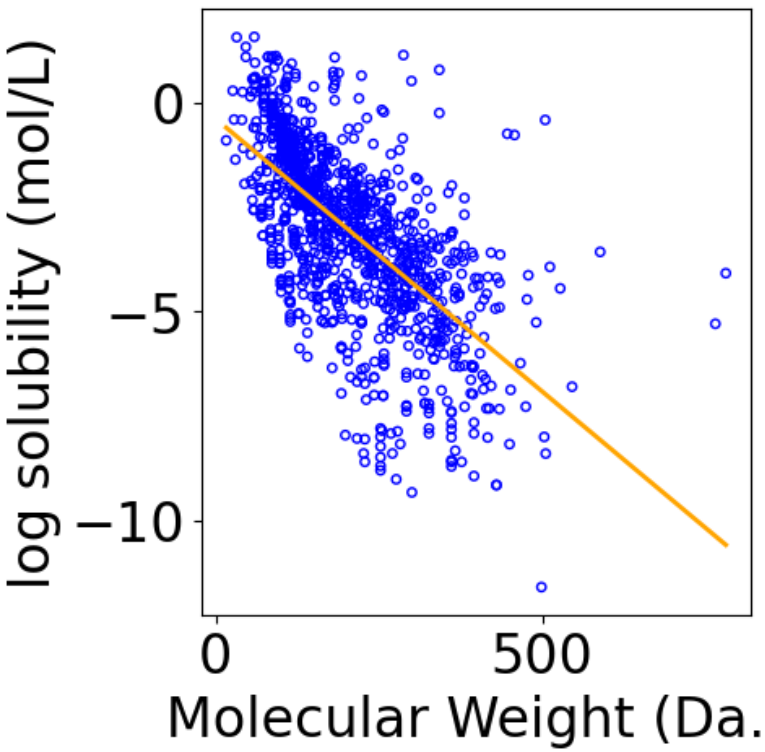
- To find the minimum of the Loss function:

$$\frac{\partial \ell(f_\theta)}{\partial \theta} = 0$$

$$\frac{\partial ||X\theta - y||_2^2}{\partial \theta}$$

$$= \frac{1}{n} 2X^T(X\theta - y) = 0$$

$$\theta = (X^T X)^{-1} X^T y$$



MSE = 2.59

The analytical solution requires that $X^T X$ is invertible - the columns of the matrix are linearly independent.

© Xuhui Huang

Gradient Descent

Numerical solutions are often used when obtaining an analytical solution is difficult (e.g., inverting a large matrix)

Gradient descent: an iterative optimization algorithm that reduces error by updating parameters in the direction of the negative gradient of the loss function, thereby minimizing it step by step.

$$-\nabla_\theta \ell(f_\theta) = -\frac{1}{n} 2X^T(X\theta - y)$$

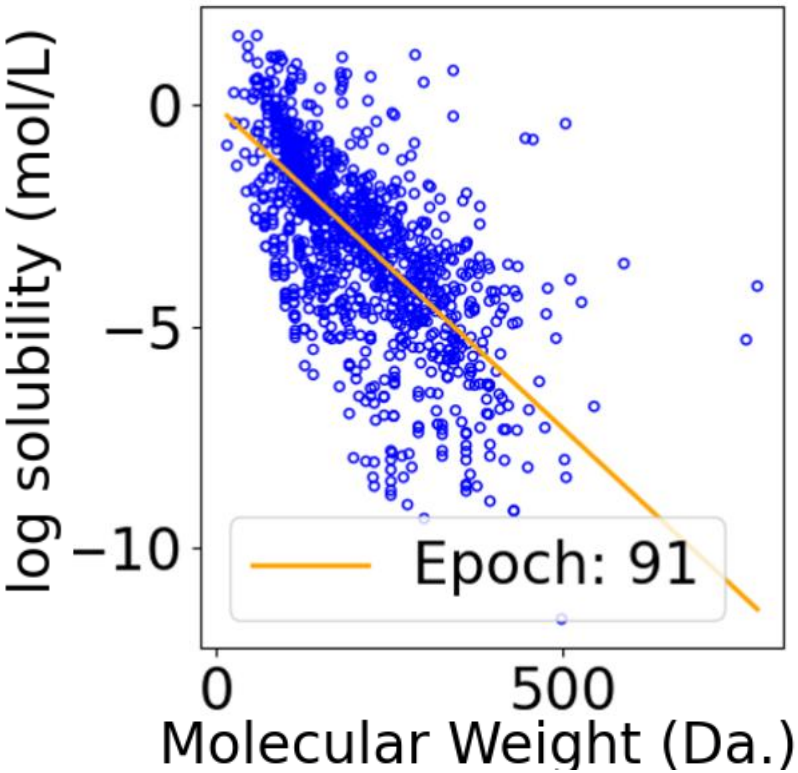
Update parameter iteratively:

$$\theta_{t+1} \leftarrow \theta_t - \alpha \cdot \nabla_\theta \ell(f_\theta)$$

$\boldsymbol{\alpha}$: learning rate.

# of steps: # of **epochs**.

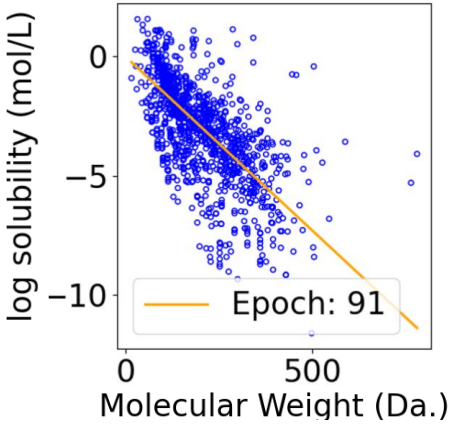
Global update: At each step (or epoch), the average of losses are computed over all data points.

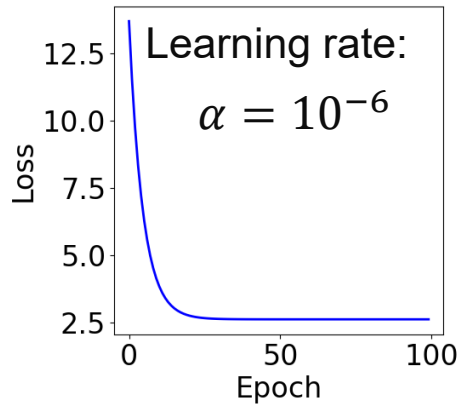


© Xuhui Huang

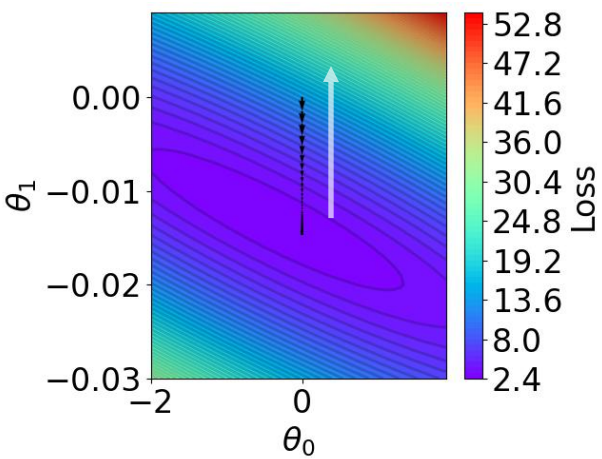
Gradient Descent

Not normalizing input data v.s. **Normalizing** input data





Best MSE = 2.62



Initial values:
$\theta_1 = 0$ & $\theta_2 = 0$

Calculate the empirical mean and standard deviation for a feature:

Mean: $$\mu_\alpha = \frac{1}{n}\sum_{i=1}^{n} x_a^{(i)}$$

Standard deviation: $$\sigma_a = \left(\frac{1}{n}\sum_{i=1}^{n}\left(x_a^{(i)} - \mu_i\right)^2\right)^{\frac{1}{2}}$$

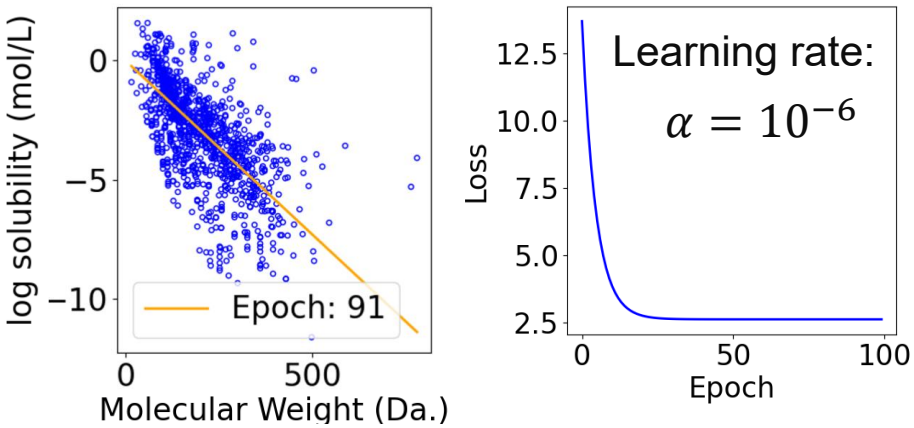
Normalize input data:

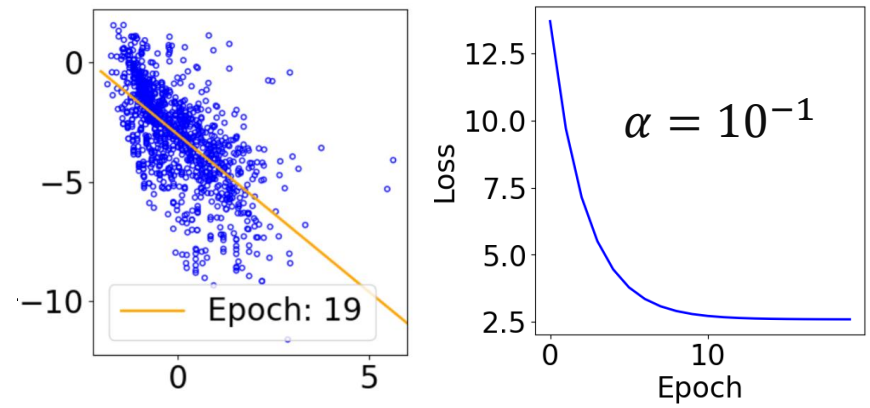
$$\tilde{x}_a^{(j)} = \frac{x_a^{(j)} - \mu_\alpha}{\sigma_a}$$

© Xuhui Huang

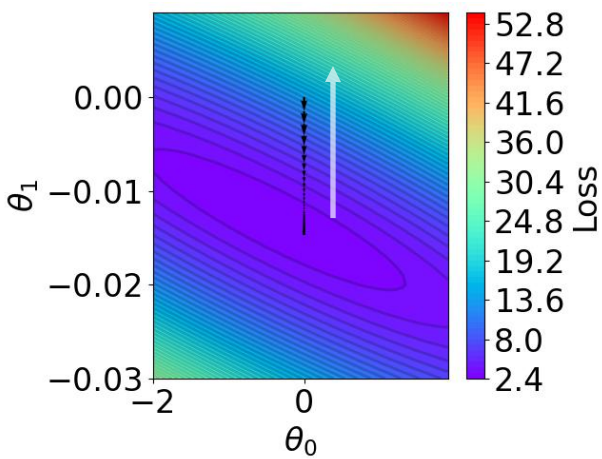
Gradient Descent

Not normalizing input data v.s. **Normalizing** input data



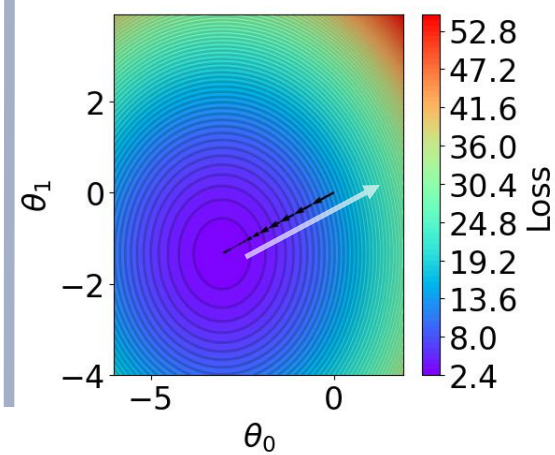


Best MSE = 2.62



Initial values:
$\theta_1 = 0$ & $\theta_2 = 0$

Best MSE = 2.59



Analytical solution:
MSE = 2.59

Normalizing the input data allows gradient descent to use a **larger learning rate** (α) and **converge in fewer epochs**.

© Xuhui Huang

Minibatch Gradient Descent

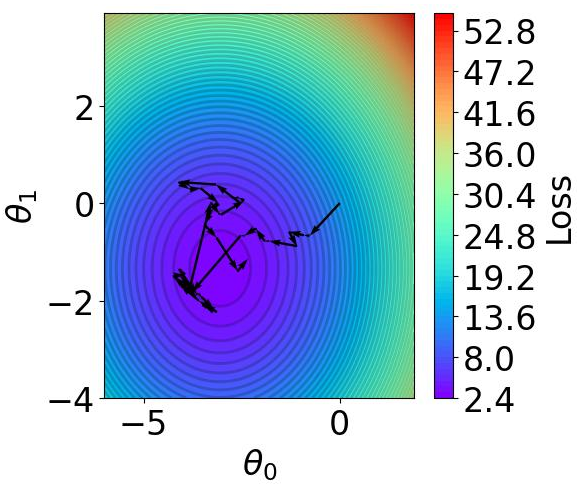
Gradient descent is very slow as it must pass over the entire dataset before making a single update.

- **Minibatch Gradient Descent**: We iteratively sample random minibatches, each containing *b* data points, to update the parameters (θ):

$$\theta_{t+1} \leftarrow \theta_t - \alpha \cdot \frac{1}{b}\left(2X_k^T X_k \theta - 2X_k^T y_k\right)$$

Stochastic GD

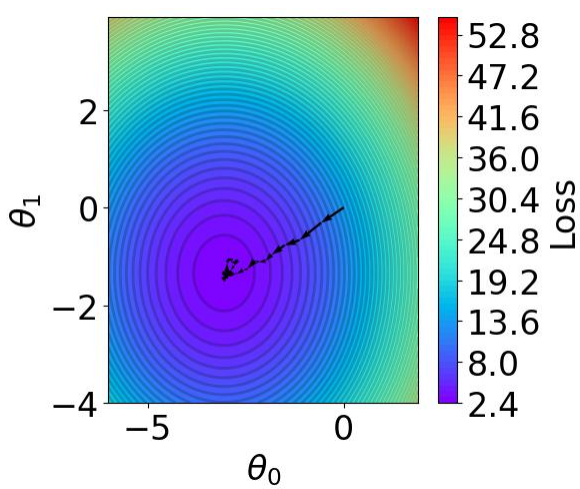
Batch size: b=1



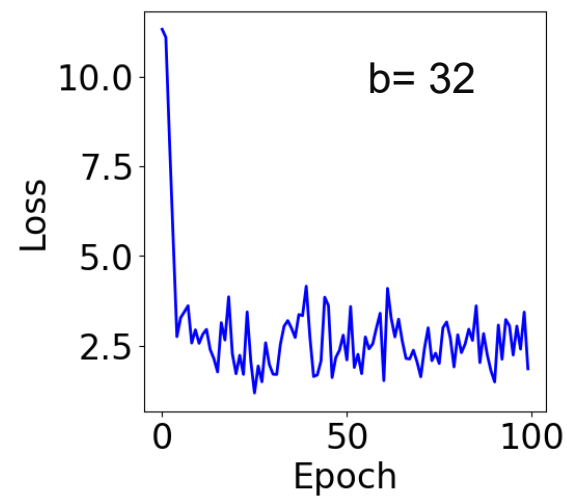
MSE=3.59

Minibatch GD

b = 32



MSE=2.59



Initial values:
$\theta_1 = 0$ & $\theta_2 = 0$

© Xuhui Huang

Linear Regression

- **Analytical Solution**: $\theta = (X^T X)^{-1} X^T y$

  Pros:

  - Accurate

  Cons:

  - $X^T X$ may not be invertible
  - Very slow

- **Gradient Descent**:

  Pros:

  Faster than the analytical solution for large datasets. The time complexity for computing the gradient at each step is:

  - $O(d^2 n)$ when using full data
  - $O(d^2 b)$ when using SGD or minibatch GD

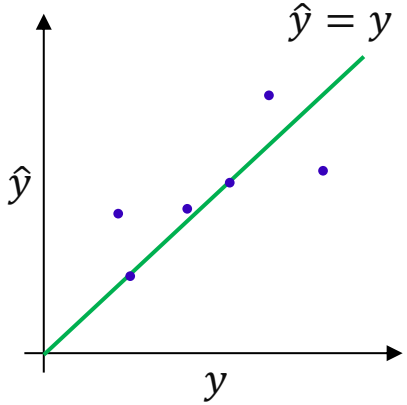
  Cons:

  - May get trapped in local minima
  - May require many epochs to converge

© Xuhui Huang

Evaluation Metrics of Regression Models

$\boldsymbol{R^2}$ (coefficient of determination) evaluates how well the model fits the data:

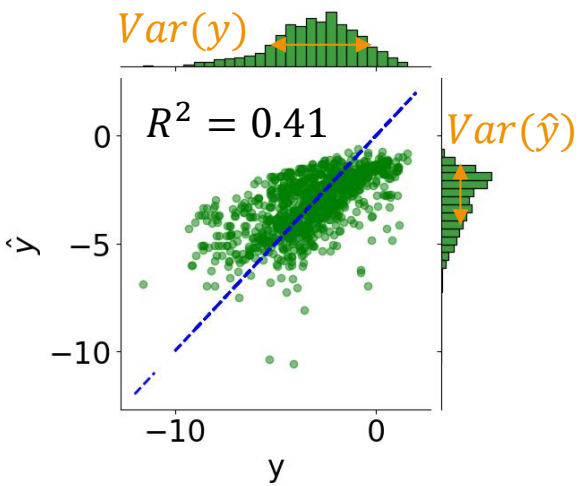


- From the view of predicted variance

$$R^2 = \frac{\sum_i (\hat{y}_i - \bar{\hat{y}})^2}{\sum_i (y_i - \bar{y})^2} = \frac{SSR}{SST} = \frac{Var(\hat{y})}{Var(y)}$$

SSR: Sum of squares regression. - Variation along the regression line.

SST: Sum of squares total. - Total variation of the data.



- From the view of noise/signal:

$$\boxed{R^2} = 1 - \boxed{\frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}} = \frac{SST - SSE}{SST}$$

$$= 1 - \frac{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}{\frac{1}{n}\sum_i (y_i - \bar{y})^2} = 1 - \frac{MSE}{Var(y)}$$

What % of variation is **not** described by the regression line.

SSE: Sum of squares error.

SST = SSR+SSE

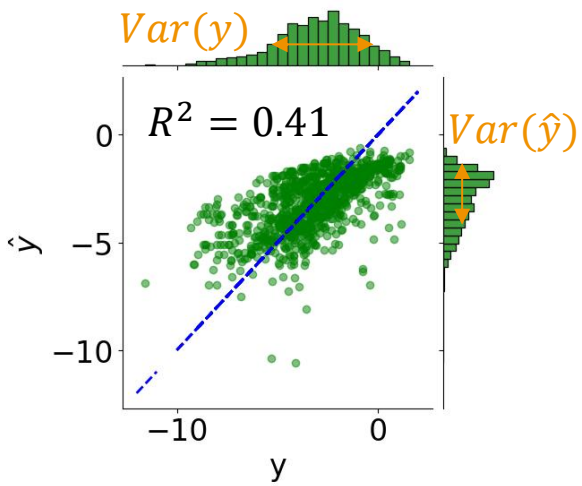
What % of total variation is described by the regression line.

© Xuhui Huang

Evaluation Metrics of Regression Models

$\boldsymbol{R^2}$ (coefficient of determination) evaluates how well the model fits the data:

$$R^2 = \frac{\sum_i (\hat{y}_i - \bar{\hat{y}})^2}{\sum_i (y_i - \bar{y})^2} = \frac{SSR}{SST} = \frac{Var(\hat{y})}{Var(y)}$$



- When $R^2 = 1$, the model perfectly explains the variance of y
- When $0 < R^2 < 1$, The model explains some but not all of the variance in y
- When $R^2 = 0$, the model predict no better than using $\bar{y}$
- When $R^2 < 0$, the model predicts worse than using $\bar{y}$

Theoretically exist, but will not happen for linear regression (it is always better than fitting to $\bar{y}$).

© Xuhui Huang

Outline

- Parametric Methods
  - Linear Regression
  - **Logistic Regression**
  - Avoiding Overfitting
- Non-parametric Methods
  - K-Nearest Neighbors (KNN)
  - Decision Trees
  - Random Forest

© Xuhui Huang

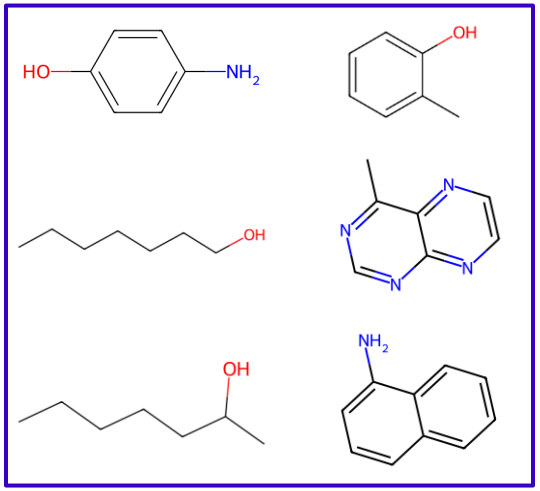
Classification

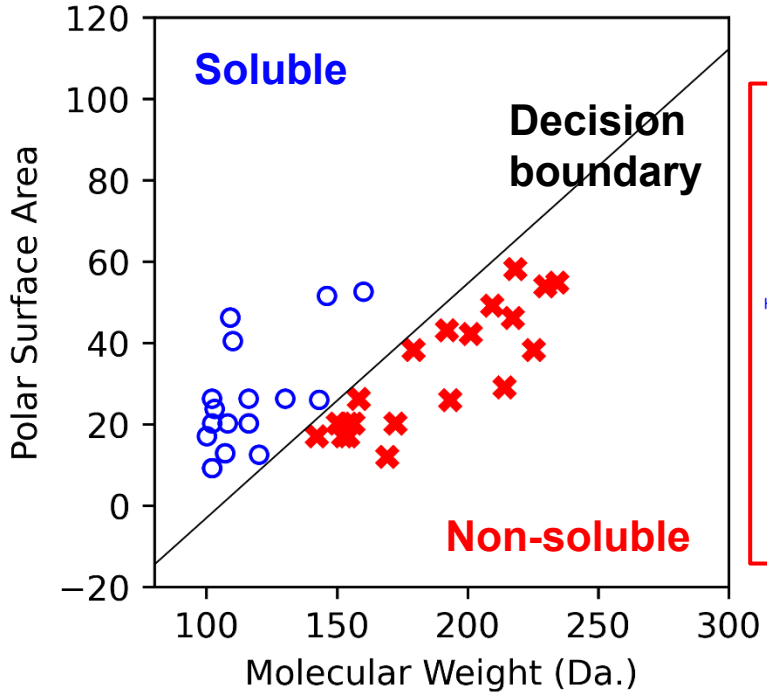
Supervised ML algorithms for classification:

Task: Training a ML model to predict **soluble** v.s. **non-soluble** compounds.

Soluble

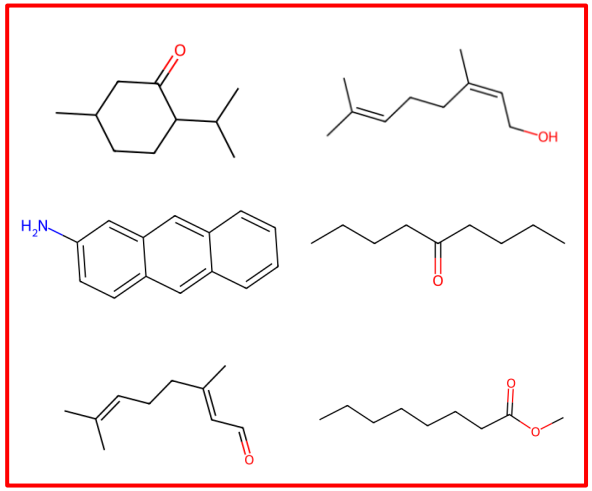
Log solubility ≥ -2





Non-soluble

Log solubility < -2



A subset of **40 compounds** from the Delaney Dataset

J. Chem. Inf. Comput. Sci. 44, 1000, (2004)

© Xuhui Huang

Linear Models for Classification

The most straightforward way – perform a linear regression, where y is -1 or 1

$$f_\theta = \theta_2 * x_2 + \theta_1 * x_1 + \theta_0$$

x_1: molecular weigh; x_2: polar surface area

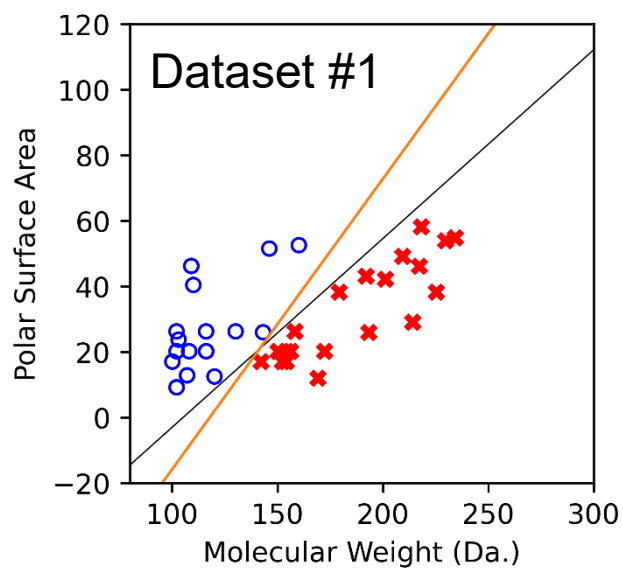
$$l(f_\theta) = \frac{1}{n}\sum_{j=1}^{n}\left(f_\theta\left(x^{(j)}\right) - y^{(j)}\right)^2$$

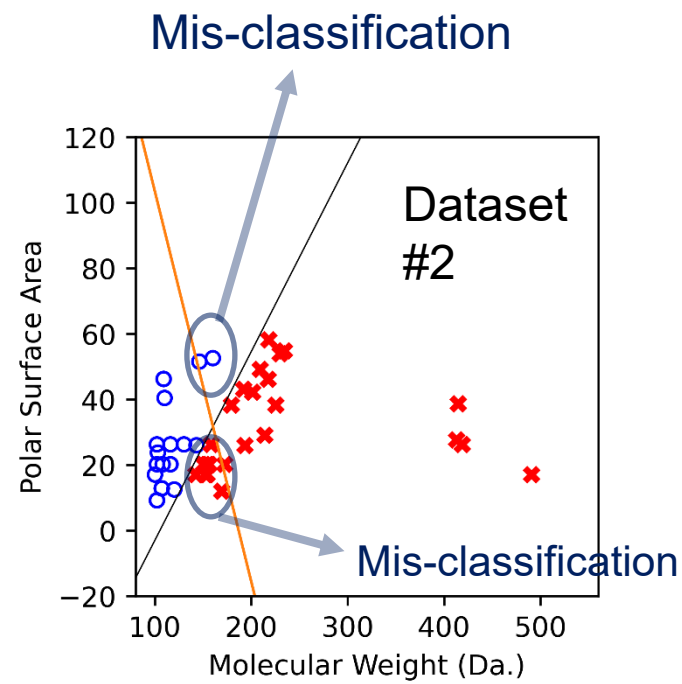
$y = 1$: Soluble – 1st Class: C_1

$y = -1$: Non-soluble – 2nd Class: C_2

Classifier function $f(x)$:

- $f_\theta(x) > 0$: Soluble, C_1
- $f_\theta(x) < 0$: Non-soluble, C_2





However, not robust to "outliers"!

© Xuhui Huang

Theoretically Optimal Classifiers

Theoretically, there exists a classifier function $f^*(x)$ that can minimize the prediction errors – minimizing the expectation of the loss function:

$$l(y, \hat{y}) = 1_{\{\hat{y} \neq y\}} = \begin{cases} 1, & \{\hat{y} = C_1, y = C_2\} \; or \; \{\hat{y} = C_2, y = C_1\} \\ 0, & \{\hat{y} = C_1, y = C_1\} \; or \; \{\hat{y} = C_2, y = C_2\} \end{cases}$$

$$\mathbb{E}[l(f\,(x), y)] = \mathbb{E}\left[1_{\{f\,(x) \neq y\}}\right] = p(f\,(x) \neq y) = R(f)$$

(*R(f)* is called risk function)

This theoretical classifier $f^*(x)$ corresponds to the underlying ground truth probabilities for the classification of data:

$$f^*(x) = \underset{C_i}{\operatorname{argmax}}\, p(y = C_i | x), \; y \in \{C_i\}_{i=1}^{k}$$

(bayes classifier)

How to estimate the ground truth probabilities $p(y = C_i | x)$ from data?

Maximum likelihood estimation (MLE)!

© Xuhui Huang

Maximum Likelihood Estimation (MLE)

We don't know the functional form of $p(y = C_i|x)$, but we can parameterize it using a function of the variable θ:

$p(y|x, \theta)$ is called the likelihood of θ

For n data points, the likelihood function is :

$$\mathcal{L}(\theta; Y, X) = \prod_{i=1}^{n} p(y_i|x_i, \theta)$$

We need to find θ that can maximize the likelihood:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmax}}\, \mathcal{L}(\theta; Y, X)$$

Or minimize negative log likelihood:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}(-\log \mathcal{L}(\theta; Y, X)) = \underset{\theta}{\operatorname{argmin}}\left(-\sum_{i=1}^{n} \log p(y_i|x_i, \theta)\right)$$

© Xuhui Huang

MLE Explained

Linear Models for Classification: how to convert discrete labels to continuous functions?

Example: A linear combination, where the true labels y = -1 or 1.

$f_\theta = \theta_2 * x_2 + \theta_1 * x_1 + \theta_0$ x_1: molecular weigh; x_2: polar surface area

$\boldsymbol{y} = \mathbf{1}$: Soluble – 1st Class: C_1 , $\boldsymbol{y} = -\mathbf{1}$: Non-soluble – 2nd Class: C_2

How do we convert continuous f_θ to the probability of a class so that we can optimize:

$$p(y_i = 1) = \mathrm{S1}(f_\theta) = \begin{cases} 100\%, & f_\theta \geq 0 \quad \text{Soluble} \\ 0.\% & f_\theta < 0 \quad \text{Non-soluble} \end{cases}$$

$$p(y_i = -1) = \mathrm{S2}(f_\theta) = \begin{cases} 0\%, & f_\theta \geq 0 \quad \text{Soluble} \\ 100\%,. & f_\theta < 0 \quad \text{Non-soluble} \end{cases}$$

Likelihood function:

$$\mathcal{L}(\theta; Y, X) = \prod_{i=1}^{n} p(y_i | x_i, \theta)$$

$$= \prod_{i=1}^{n} \left(\mathbf{1}(y_i = 1) * \mathrm{S1}(f_\theta) + \mathbf{1}(y_i = -1) * \mathrm{S2}(f_\theta) \right)$$

© Xuhui Huang

MLE Explained

Linear Models for Classification

Example: A linear combination, where the true labels y = -1 or 1.

$$f_\theta = \theta_2 * x_2 + \theta_1 * x_1 + \theta_0$$

x_1: molecular weigh; x_2: polar surface area

$\boldsymbol{y} = \mathbf{1}$: Soluble – 1st Class: C_1 , $\boldsymbol{y} = -\mathbf{1}$: Non-soluble – 2nd Class: C_2

Compound 1:

$$f_{\theta_1} = 0.1, y_1 = 1$$

Compound 2:

$$f_{\theta_2} = -0.2, y_2 = 1$$

Compound 3:

$$f_{\theta_3} = -0.1, y_3 = -1$$

© Xuhui Huang

MLE Explained

Linear Models for Classification

Example: A linear combination, where the true labels y = -1 or 1.

$f_\theta = \theta_2 * x_2 + \theta_1 * x_1 + \theta_0$ x_1: molecular weigh; x_2: polar surface area

$y = 1$: Soluble – 1st Class: C_1 , $y = -1$: Non-soluble – 2nd Class: C_2

Compound 1:

$f_{\theta_1} = 0.1, y_1 = 1$

$S1(f_{\theta_1}) = 1,$

Compound 2:

$f_{\theta_2} = -0.2, y_2 = 1$

$S1(f_{\theta_2}) = 0,$

Compound 3:

$f_{\theta_3} = -0.1, y_3 = -1$

$S2(f_{\theta_3}) = 1$

$\mathcal{L}(\theta; Y, X) = 1 * 0 * 1 = 0$

© Xuhui Huang

MLE Explained

Linear Models for Classification

We don't know the functional form of $p(y = C_i|x)$, but we can parameterize it using a function of the variable θ:

$p(y|x, \theta)$ is called the likelihood of θ

Maximize the likelihood function:

$$\mathcal{L}(\theta; Y, X) = \prod_{i=1}^{n} p(y_i|x_i, \theta)$$

n data points

Minimize the -log likelihood function:

$$-\sum_{i=1}^{n} \log p(y_i|x_i, \theta)$$

© Xuhui Huang

Exercise

Consider tossing a coin where the outcomes are heads (1) and tails (0). Suppose you toss the coin 10 times, and the results are $Y = \{1,1,0,0,1,0,0,1,1,1\}$.

You aim to use Maximum Likelihood Estimation (MLE) to estimate $p(y = 1)$. Specifically, you parameterize $p(y = 1|x, \theta) = \theta$ and thus $p(y = 0|x, \theta) = 1 - \theta$

What is the estimated $\hat{p}(y = 1)$ in this case?

© Xuhui Huang

Exercise

Consider tossing a coin where the outcomes are heads (1) and tails (0). Suppose you toss the coin 10 times, and the results are $Y = \{1,1,0,0,1,0,0,1,1,1\}$.

We aim to use Maximum Likelihood Estimation (MLE) to estimate $p(y = 1)$. Specifically, we parameterize $p(y = 1|x, \theta) = \theta$ and thus $p(y = 0|x, \theta) = 1 - \theta$

What is the estimated $\hat{p}(y = 1)$ in this case?

$$\mathcal{L}(\theta; Y, X) = p(Y|X, \theta) = \theta^6(1-\theta)^4$$

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}(-\log \mathcal{L}(\theta; Y)) = \underset{\theta}{\operatorname{argmin}}(-6 * \log\theta - 4 * \log(1-\theta))$$

$$\left.\frac{d(-6 * \log\theta - 4 * \log(1-\theta))}{d\theta}\right|_{\theta=\hat{\theta}} = -\frac{6}{\hat{\theta}} + \frac{4}{1-\hat{\theta}} = 0$$

$$\hat{\theta} = 0.6$$

© Xuhui Huang

Logistic Regression

Logistic regression uses sigmoid function to parameterize $p(y_i|x_i,\theta)$ for binary classification:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\left(-\sum_{i=1}^{n} \log p(y_i|x_i,\theta)\right)$$

For soluble v.s. non-soluble prediction:

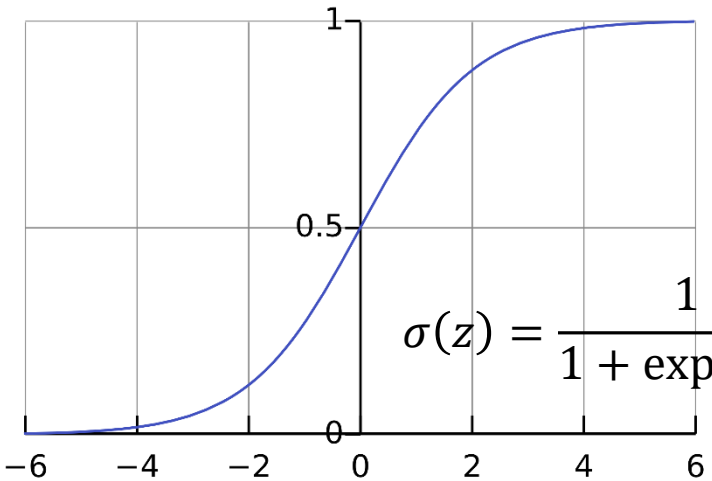
$$f_\theta = \theta^T x = \theta_2 * x_2 + \theta_1 * x_1 + \theta_0$$

x_1: molecular weigh; x_2: polar surface area

$y = 1$: Soluble, $y = -1$: Non-soluble

$$p(y=1|x,\theta) = \sigma(\theta^T x) = \frac{1}{1+\exp(-\theta^T x)}$$

$$p(y=-1|x,\theta) = 1 - \sigma(\theta^T x) = \frac{1}{1+\exp(\theta^T x)}$$



$$\sigma(z) = \frac{1}{1+\exp(-z)}$$

Therefore:

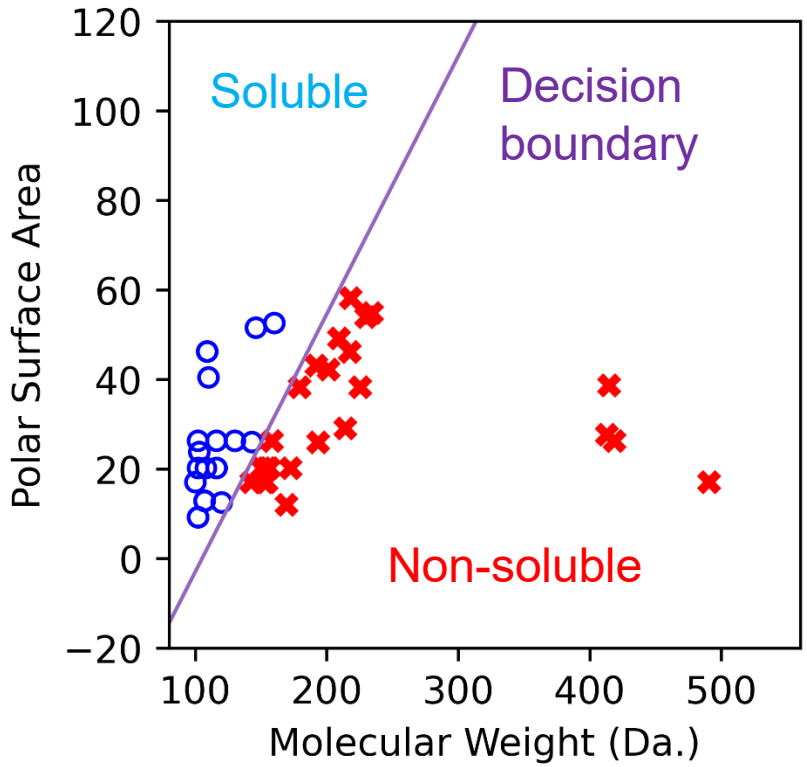
$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\left(-\sum_{y_i=1} \log \sigma(\theta^T x_i) - \sum_{y_i=0} \log\big(1-\sigma(\theta^T x_i)\big)\right)$$

© Xuhui Huang

Logistic Regression

No analytical solution, and apply gradient descent to optimize:

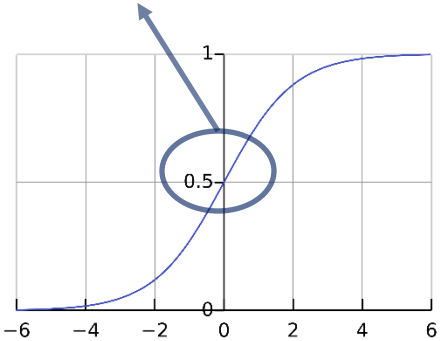
$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \left(-\sum_{y_i=1} \log \sigma(\theta^T x_i) - \sum_{y_i=0} \log\left(1 - \sigma(\theta^T x_i)\right) \right)$$



$$p(y = 1|x, \theta) = 0.5$$

or

$$f_\theta = \theta^T x = 0$$



© Xuhui Huang

Evaluation Metrics for Classification Task

How can we evaluate the performance of a classification model?

Can we use the loss function defined in logistic regression?

$$Loss = -\sum_{y_i=1} \log \sigma(\theta^T x_i) - \sum_{y_i=0} \log\big(1 - \sigma(\theta^T x_i)\big)$$

The loss function is not straightforward for assessing model quality and varies across methods!

The most common approach:

Model Accuracy - the proportion of correctly classified samples

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

© Xuhui Huang

Evaluation Metrics for Classification Task

Accuracy and the confusion matrix (2-class):

		Actual class: positive	Actual class: negative
Predicted class	positive	**True positives (TP)**	**False positives (FP)**
	negative	**False negatives (FN)**	**True negatives (TN)**

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} = \frac{TP+TN}{TP+TN+FP+FN}$$

© Xuhui Huang

Evaluation metrics for classification task

Class-imbalanced datasets pose challenges for “Accuracy” measurement.

Example: Predict a compound’s solubility *(soluble: positive* or *non-soluble: negative)* from a dataset of 1000 compounds, including 10 soluble (positive) ones.

We have the following model prediction:

		Actual class: positive	Actual class: negative
Predicted class	positive	**TP=0**	**FP=40**
	negative	**FN=10**	**TN=950**

$$\textbf{Accuracy} = \frac{\mathrm{TP+TN}}{\mathrm{TP+TN+FP+FN}} = \frac{0+950}{0+10+40+950} = 95\%$$

The model correctly predicts **0** soluble compounds but still achieves 95% accuracy!

© Xuhui Huang

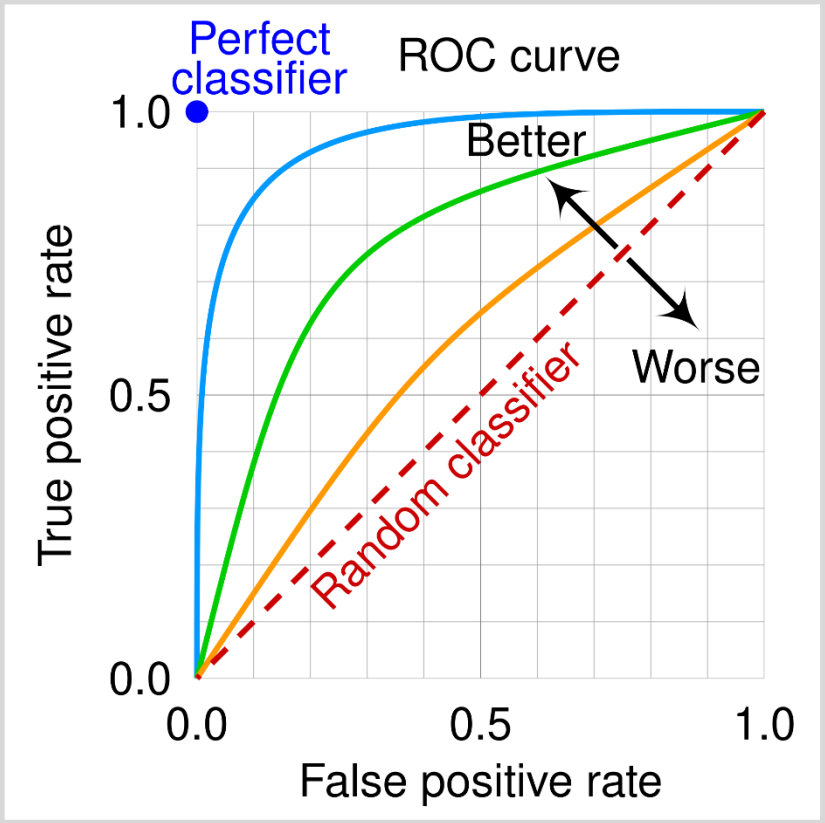
Receiver Operating Characteristic (ROC) Curve

ROC curve represents a classifier's performance by plotting the **true positive rate** v.s. **false positive rate** as a function of a classifier parameter.

True positives (TP)	**False positives (FP)**
False negatives (FN)	**True negatives (TN)**

Y axis: True positive rate (recall) = $\frac{TP}{TP+FN}$

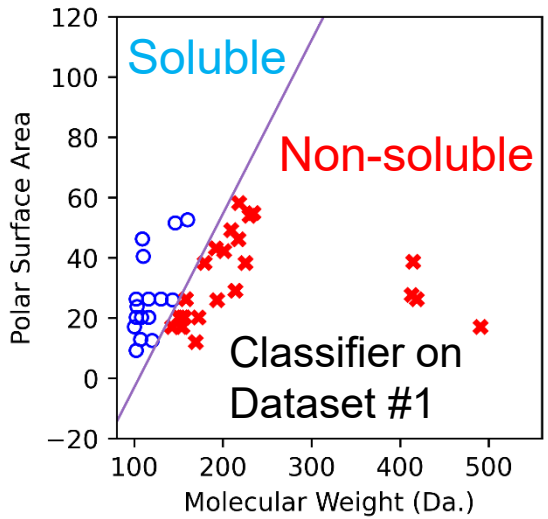
X axis: False positive rate = $\frac{FP}{FP+TN}$

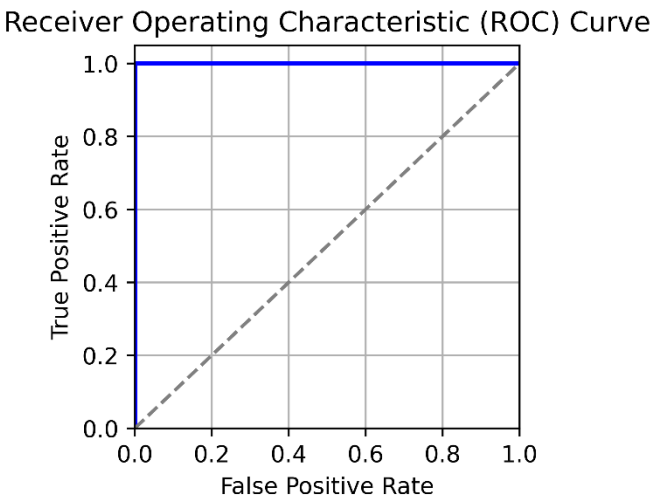


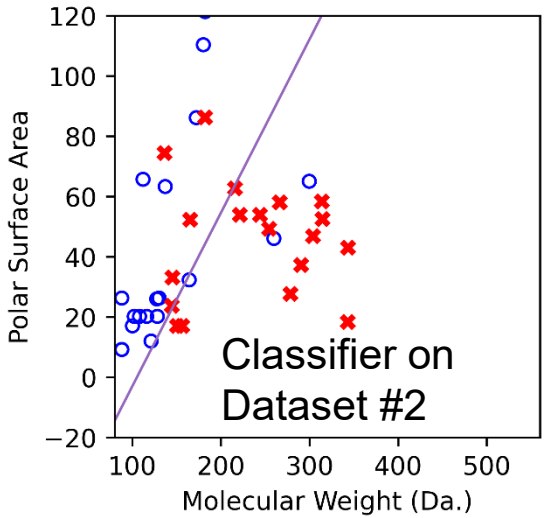
https://en.wikipedia.org/

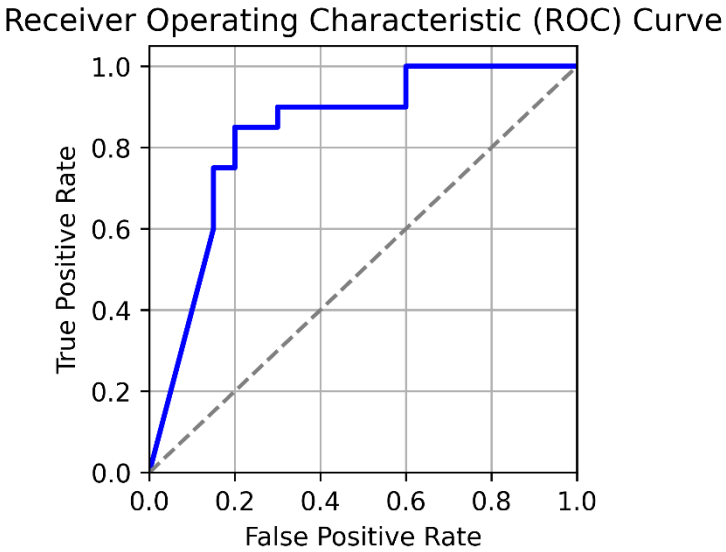
© Xuhui Huang

Receiver Operating Characteristic (ROC) Curve









Based on ROC curves, Classifier #1 outperforms Classifier #2 in predicting soluble v.s. non-soluble compounds.

© Xuhui Huang

Each Point on ROC corresponds to a Confusion Matrix

Example of ROC curve: True positive rate vs. false positive rate as a function of a classifier parameter - decision boundary threshold.

Decision boundary (threshold at $f_\theta(x) = 0$)

Compound ID	f_θ	True Label
Comp #5	0.90	1
Comp #2	0.50	1
Comp #1	0.30	-1
Comp #3	0.20	1
Comp #7	0.10	1
Comp #6	-0.60	-1
Comp #4	-0.90	-1

TP = 4, FP=1, FN=0, TN=2

True positive rate: 4/(4+0)=1

False positive rate: 1/(1+2)=1/3

	Actual class: positive	Actual class: negative
Predicted class: positive	**True positives (TP)**	**False positives (FP)**
Predicted class: negative	**False negatives (FN)**	**True negatives (TN)**

- TP+FN always equals to 4 – based on true labels.
- FP+TN always equals to 3 – based on true labels.

© Xuhui Huang

Plot ROC

Example of ROC curve: True positive rate vs. false positive rate as a function of a classifier parameter - decision boundary threshold.

Move the decision boundary:

Compound ID	f_θ	True Label
Comp #5	0.90	1
Comp #2	0.50	1
Comp #1	0.30	-1
Comp #3	0.20	1
Comp #7	0.10	1
Comp #6	-0.60	-1
Comp #4	-0.90	-1

Threshold >=0.90, TP = 1/4, FP = 0/3

f_θ >=0.90, “Soluble”, Predicted label: +1

f_θ <0.90, “Non-soluble”, Predicted label: -1

- TP+FN always equals to 4
- FP+TN always equals to 3

© Xuhui Huang

Plot ROC

Example of ROC curve: True positive rate vs. false positive rate as a function of a classifier parameter - decision boundary threshold.

Move the decision boundary:

Compound ID	f_θ	**True Label**
Comp #5	0.90	1
Comp #2	0.50	1
Comp #1	0.30	-1
Comp #3	0.20	1
Comp #7	0.10	1
Comp #6	-0.60	-1
Comp #4	-0.90	-1

Threshold >=0.90, TP = 1/4, FP = 0/3

Threshold >=0.50, TP = 2/4, FP = 0/3

- TP+FN always equals to 4
- FP+TN always equals to 3

© Xuhui Huang

Plot ROC

Example of ROC curve: True positive rate vs. false positive rate as a function of a classifier parameter - decision boundary threshold.

Move the decision boundary:

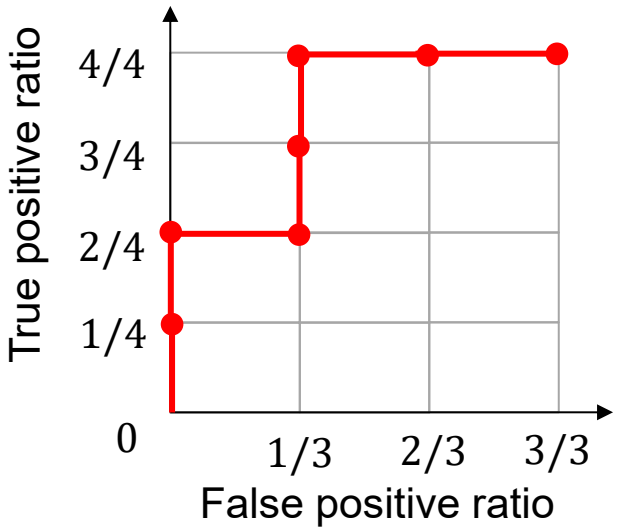
Compound ID	f_θ	True Label
Comp #5	0.90	1
Comp #2	0.50	1
Comp #1	0.30	-1
Comp #3	0.20	1
Comp #7	0.10	1
Comp #6	-0.60	-1
Comp #4	-0.90	-1

Threshold >=0.90, TP = 1/4, FP = 0/3

Threshold >=0.50, TP = 2/4, FP = 0/3

Threshold >=0.30, TP = 2/4, FP = 1/3

….



- TP+FN always equals to 4
- FP+TN always equals to 3

© Xuhui Huang

Outline

- Parametric Methods
  - Linear Regression
  - Logistic Regression
  - **Avoiding Overfitting**
- Non-parametric Methods
  - K-Nearest Neighbors (KNN)
  - Decision Trees
  - Random Forest

© Xuhui Huang

Generalization: Predicting Previously Unseen Data

ML: **Featurization** → **Training (Learning)** → **Generation (Prediction)**

Example: Predicting aqueous solubility using a Polynomial function on the Delaney dataset.

$$\hat{y}(x,\theta) = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_M x^M = \sum_{j=1}^{M} \theta_j \, x^j$$

Error function:

$$\ell(\theta) = \frac{1}{2}\sum_{n=1}^{N} (\hat{y}(x_n,\theta) - y_n)^2$$

1128 compounds:

Name	logS (mol/L)	Molecular Weight (Da.)
diazinon	-3.64	304.352
parathion	-4.66	291.265
…	…	…

Training	Test

- **Split** data into training and test sets.
- **Train** the model on the training set.
- **Evaluate** the model's generalization error on the test data (unseen during training).

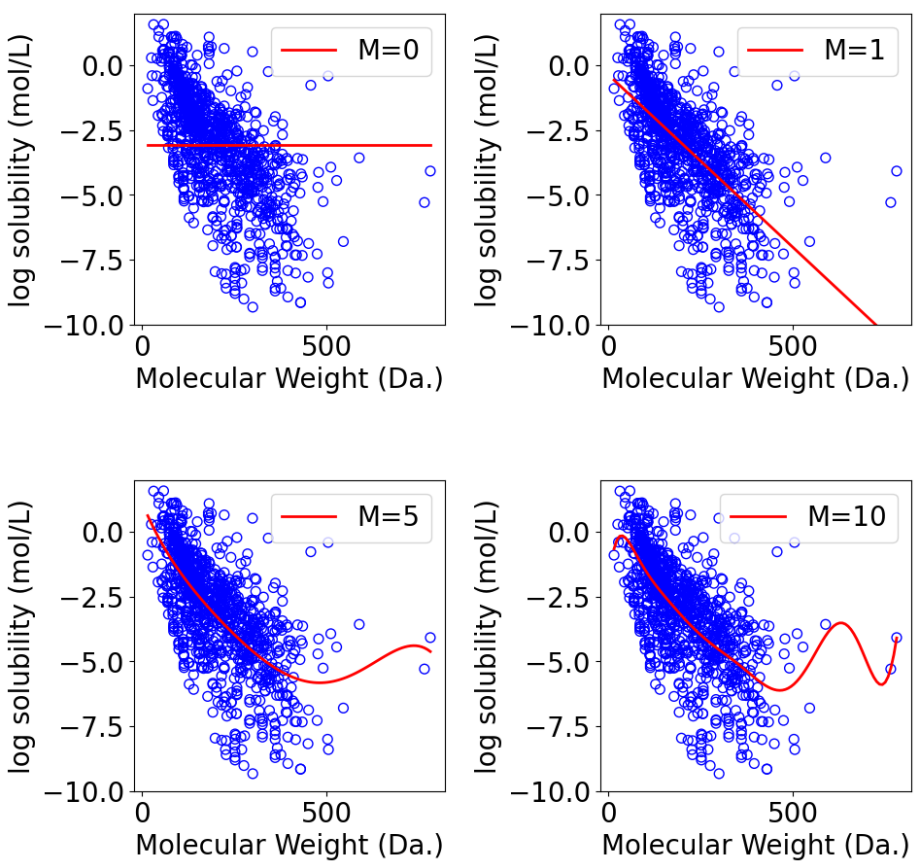
Delaney, ***J. Chem. Inf. Comput. Sci.*** 44, 1000, (2004)

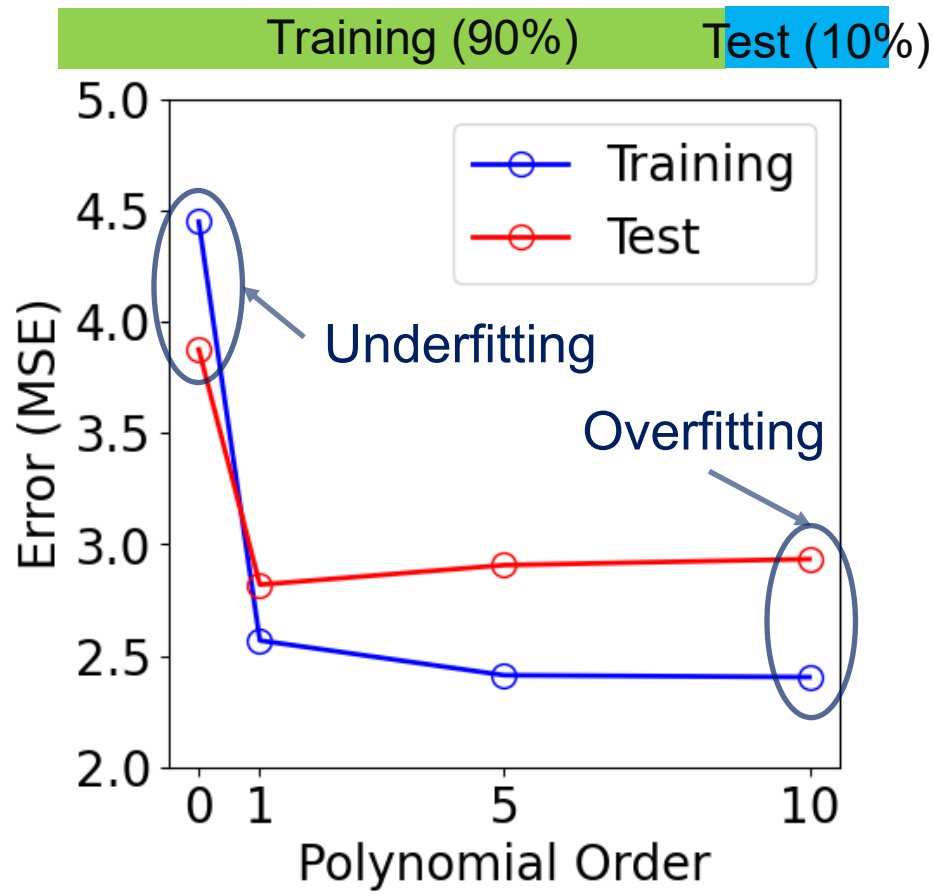
© Xuhui Huang

Underfitting and Overfitting

Polynomial fitting to the Delaney dataset:

$$\hat{y}(x,\theta) = \sum_{j=1}^{M} \theta_j \, x^j$$



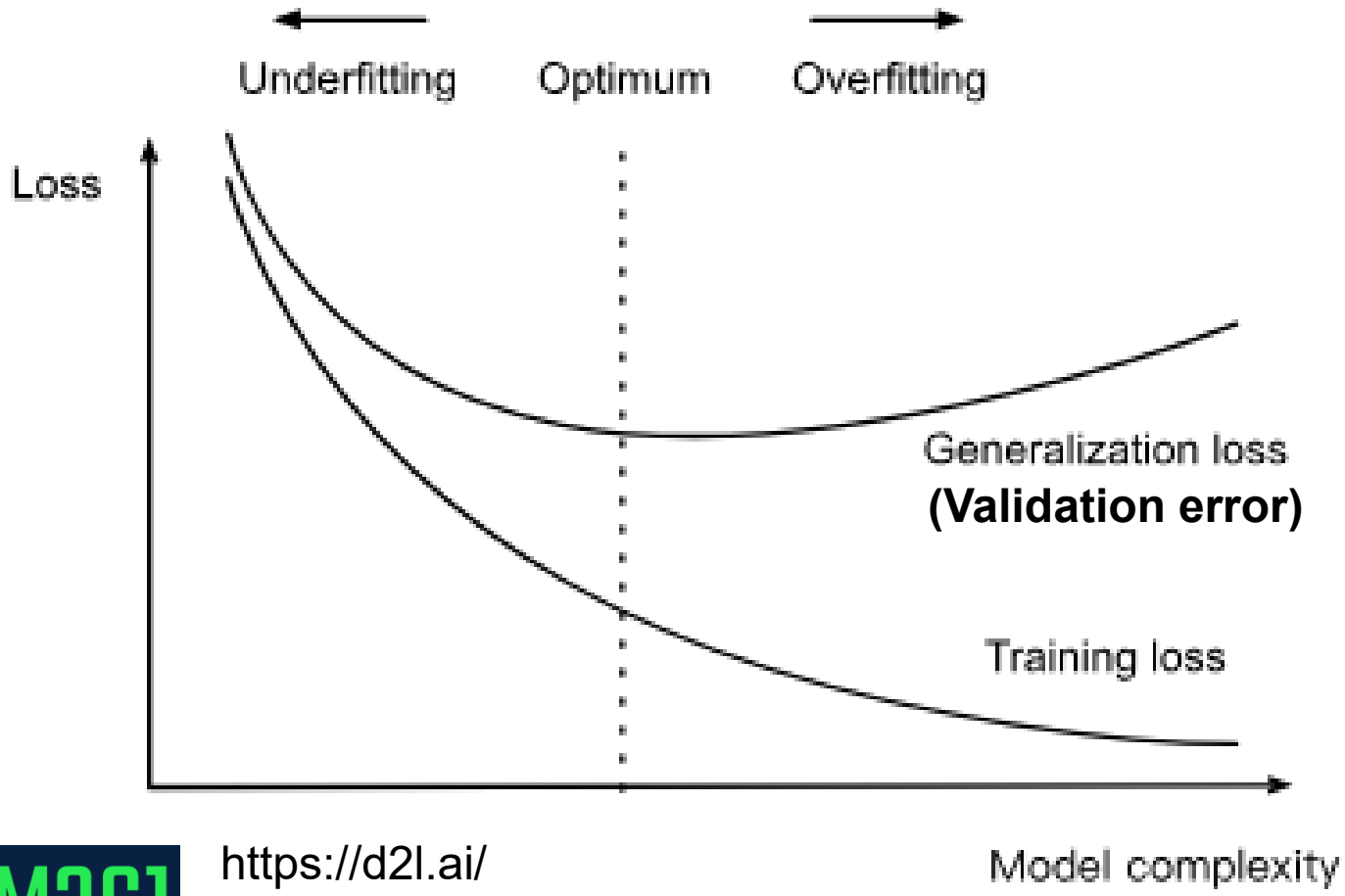


© Xuhui Huang

Underfitting and Overfitting

Underfitting: Both training and generalization errors are high and with a small gap. The model is too simple to capture the data pattern and further reduce training error.

Overfitting: The training error is substantially lower than the generalization/validation error.



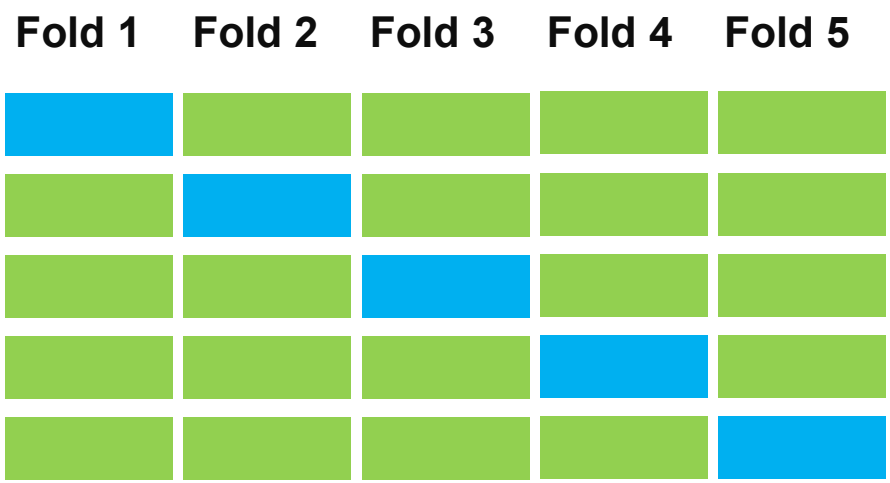
https://d2l.ai/

© Xuhui Huang

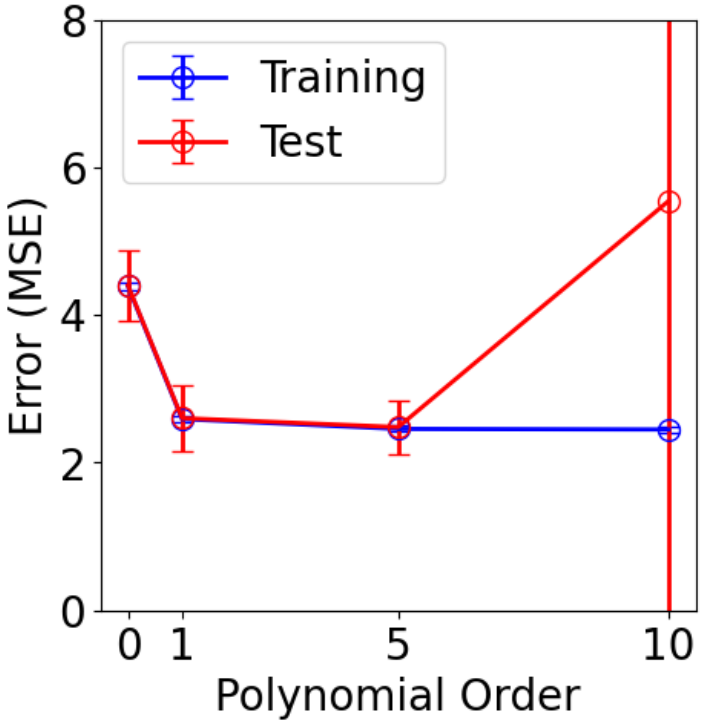
K-Fold Cross-Validation

A single train-test split can cause high variance as performance depends on data allocation. K-fold cross-validation mitigates this issue.

K-Fold cross-validation



10-Fold Cross-Validation for solubility prediction



$M = 10$ overfits the data

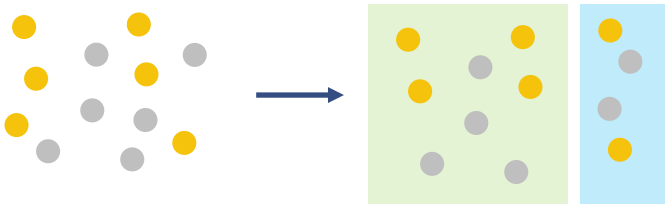
$M = 0$ underfits the data

$M = 5$ is the best choice

© Xuhui Huang

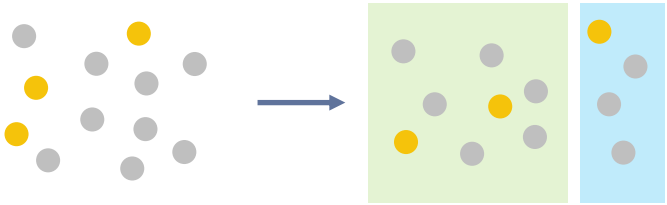
Data Split Strategies

Random Split



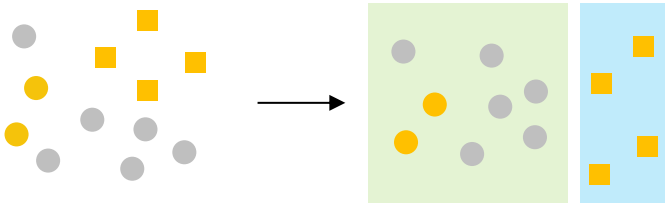
- The distribution of training data and test data are similar under random split

Stratified Split



- A stratified split partitions a dataset while preserving class proportions in each subset. Good for imbalanced dataset!

Split by cluster



- First, cluster the dataset and then split by clusters. This approach assesses the model's performance in predicting dissimilar compounds.

© Xuhui Huang

Regularization to Combat Overfitting

Ridge regularization adds the L2 norm of the weight vector (θ) as a penalty (regularization term) to the loss function.

L2 norm: $||\theta||_2 = \sqrt{\sum_{i=1}^{n} \theta_i^2}$

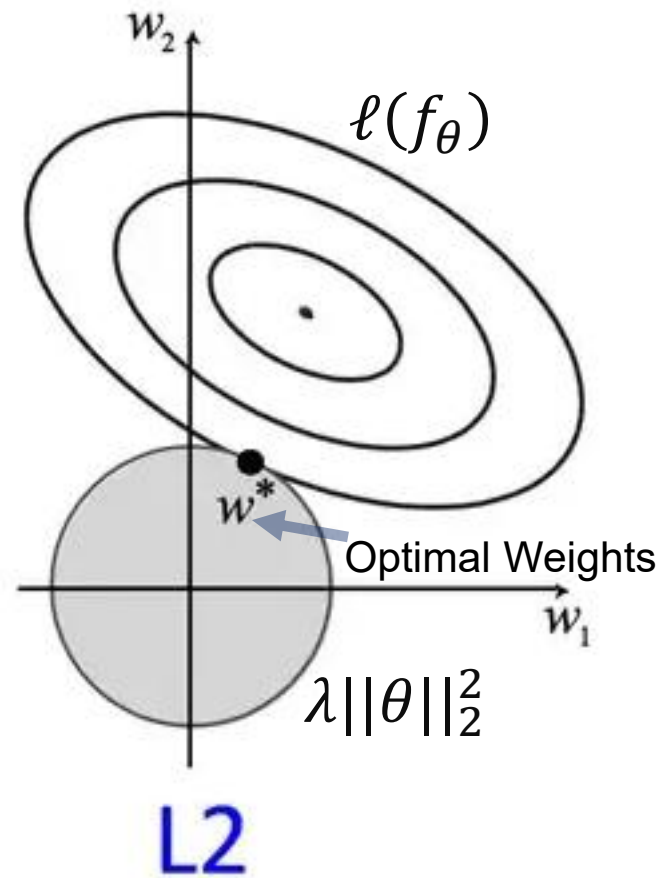
$$\ell'(f_\theta) = \ell(f_\theta) + \lambda||\theta||_2^2$$

$$\nabla_\theta \ell'(f_\theta) = \nabla_\theta \ell(f_\theta) + 2\lambda\theta$$

Gradient descent:

$$\theta_{t+1} \leftarrow \theta_t - \alpha\nabla_\theta \ell(f_\theta) - 2\alpha\lambda\theta$$

Weight decay: The penalty term reduces the magnitude of the weights at each training step, preventing them from growing too large.



© Xuhui Huang

Regularization to Combat Overfitting

Lasso regularization adds the L1 norm of the weight vector (θ) as a penalty (regularization term) to the loss function.

$$\ell'(f_\theta) = \ell(f_\theta) + \lambda||\theta||_1$$

L1 norm: $||\theta||_1 = \sum_{i=1}^{n} |\theta_i|$

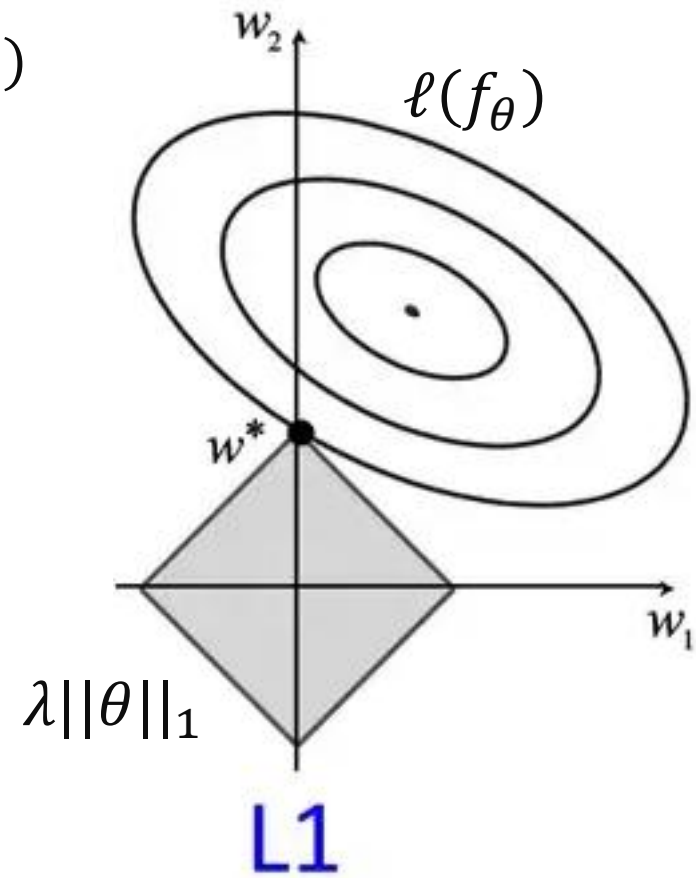
$$\nabla_\theta \ell'(f_\theta) = \nabla_\theta \ell(f_\theta) + \lambda \cdot sign(\theta)$$

$$sign(\theta_i) = \begin{cases} -1, \theta_i < 0 \\ 0, \theta_i = 0 \\ 1, \theta_i > 0 \end{cases}$$

Gradient descent:

$$\theta_{t+1} \leftarrow \theta_t - \alpha \cdot \nabla_\theta \ell(f_\theta) - \alpha\lambda \cdot sign(\theta)$$

Weight decay

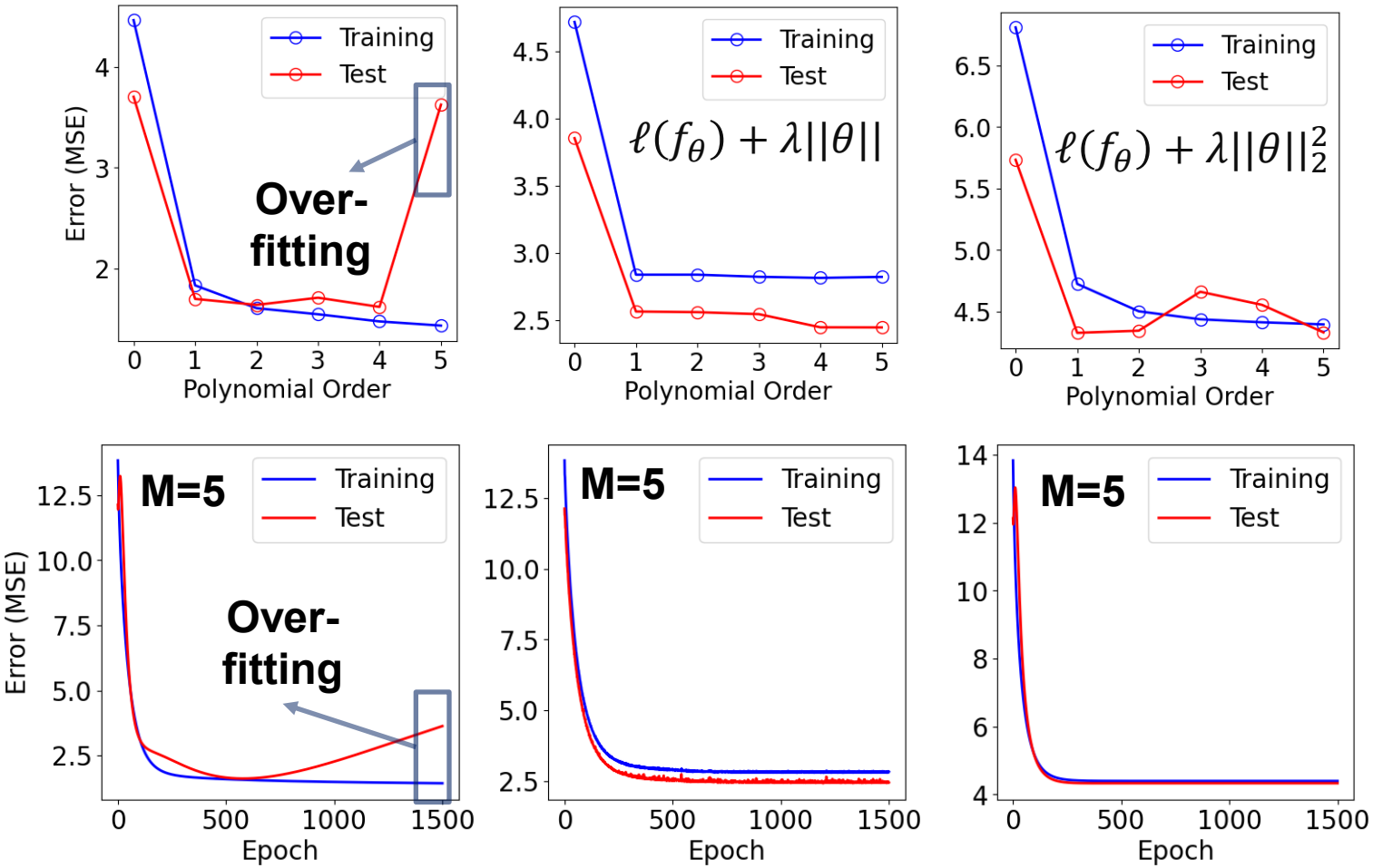


© Xuhui Huang

Regularization to Combat Overfitting

Polynomial fitting to the Delaney dataset. Features: [MW, Hdonor, Rings, Rotatable Bonds].

No regularization | **Lasso (L1 norm)** | **Ridge (L2 norm)**



Regularization helps prevent overfitting, especially in complex models.

© Xuhui Huang

Exercise

We train a linear regression model to predict compound solubility using five input features. Without regularization, the optimal weights are:

$$\theta^* = [-3 \quad -1 \quad -0.5 \quad -0.06 \quad 0.2]^T$$

Below are the optimal weights with Ridge or Lasso regularization applied. Could you match each set to the corresponding method?

(a) $\theta_a^* = [-1.5 \quad -0.50 \quad -0.30 \quad -0.14 \quad 0.10]^T$

(b) $\theta_b^* = [-2.5 \quad -0.8 \quad 0 \quad 0 \quad 0]^T$

© Xuhui Huang

Exercise

We train a linear regression model to predict compound solubility using five input features. Without regularization, the optimal weights are:

$$\theta^* = [-3 \quad -1 \quad -0.5 \quad -0.06 \quad 0.2]^T$$

Below are the optimal weights with Ridge or Lasso regularization applied. Could you match each set to the corresponding method?

(a) $\theta_a^* = [-1.5 \quad -0.50 \quad -0.30 \quad -0.14 \quad 0.10]^T$ **- Ridge regularization**

- The weights underwent decay

(b) $\theta_b^* = [-2.5 \quad -0.8 \quad 0 \quad 0 \quad 0]^T$ **- Lasso regularization**

- Weight decay, and **several weights were reduced to zero.**

© Xuhui Huang

Exercise

We train a linear regression model to predict compound solubility using five input features. Without regularization, the optimal weights are:

$$\theta^* = [-3 \quad -1 \quad -0.5 \quad -0.06 \quad 0.2]^T$$

Below are the optimal weights with Ridge or Lasso regularization applied. Could you match each set to the corresponding method?

(a) $\theta_a^* = [-1.5 \quad -0.50 \quad -0.30 \quad -0.14 \quad 0.10]^T$ **- Ridge regularization**

- The weights underwent decay

(b) $\theta_b^* = [-2.5 \quad -0.8 \quad 0 \quad 0 \quad 0]^T$ **- Lasso regularization**

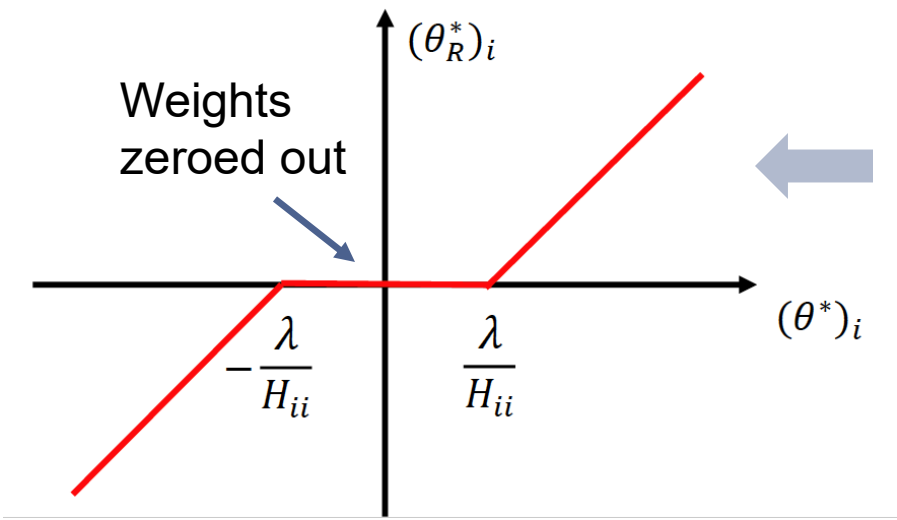
- Weight decay, and **several weights were reduced to zero.**

© Xuhui Huang

Exercise

(b) $\theta_b^* = [-2.5 \quad -0.8 \quad 0 \quad 0 \quad 0]^T$ **- Lasso regularization**

- Weight decay, and **several weights were reduced to zero.**



We can derive an approximate analytical solution for Lasso: θ_R^*.

$\boldsymbol{\theta_R^*}$ (Lasso optimal weights) **v.s.** $\boldsymbol{\theta}^*$ (optimal weights without regularization).

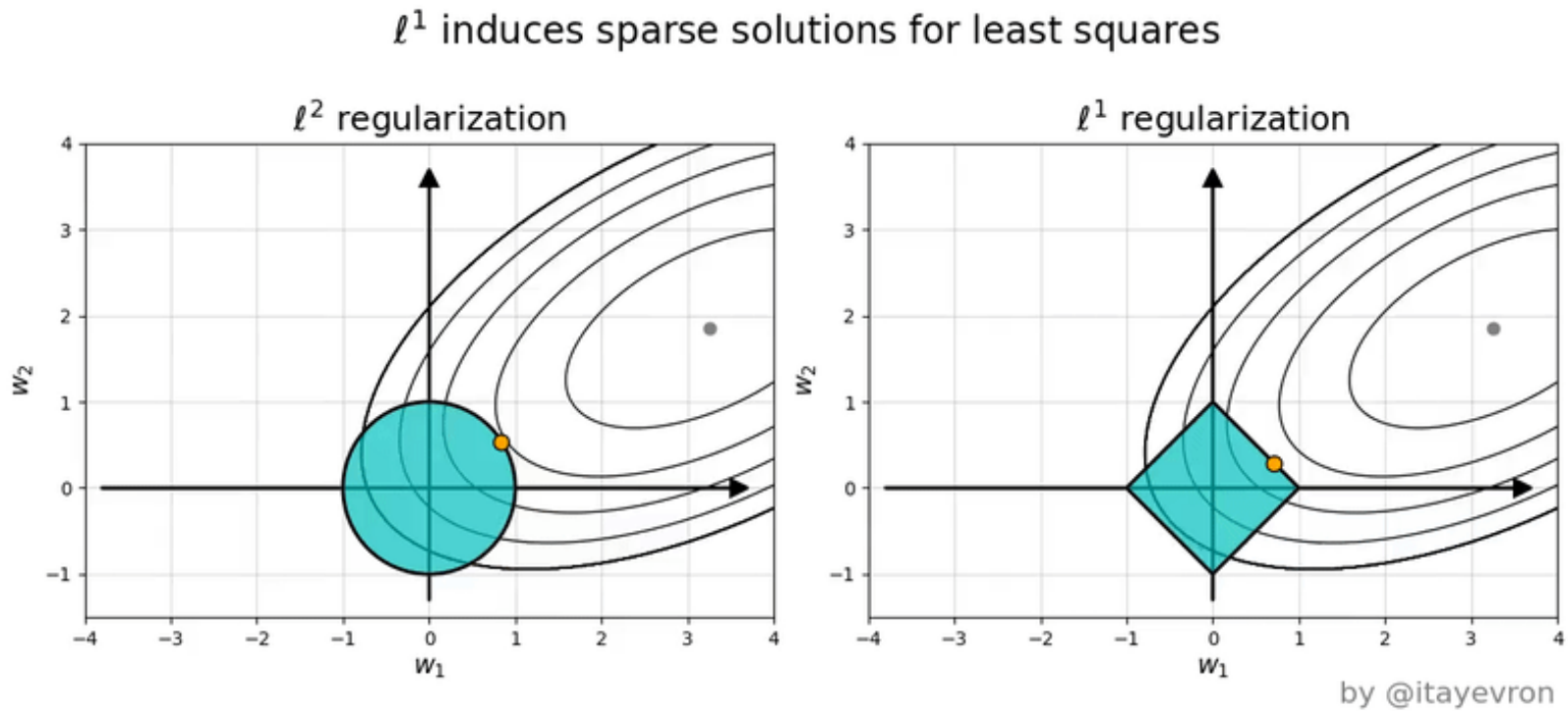
Lasso induces **sparsity** in its optimized weights!

H: the Hessian matrix of the original loss function: $\ell(f_\theta)$.

The Lasso loss function: $\ell'(f_\theta) = \ell(f_\theta) + \lambda||\theta||_1$.

© Xuhui Huang

Lasso (L1 norm) v.s. Ridge (L2 norm)



The L1 regularization favors weights aligned with either the x- or y-axis (with zero on the other axis) and discourages staying in between, unlike the L2 regularization.

https://satishkumarmoparthi.medium.com/

© Xuhui Huang

Lasso (L1 norm) v.s. Ridge (L2 norm)

- The regularization term penalizes large parameters. Both Lasso and Ridge regularization shrink parameters toward zero.

- Generally, Lasso produces sparser models with more zeroed-out parameters, while Ridge results in smaller but nonzero parameters.

- Regularization slightly increases bias (bad) but reduces variance (good). The hyperparameter λ can be tuned to achieve a tradeoff between bias and variance.

Code demonstration for Logistic regression

© Xuhui Huang

Outline

- Parametric Methods
  - Linear Regression
  - Logistic Regression
  - Avoiding Overfitting
- **Non-parametric Methods**
  - **K-Nearest Neighbors (KNN)**
  - Decision Trees
  - Random Forest

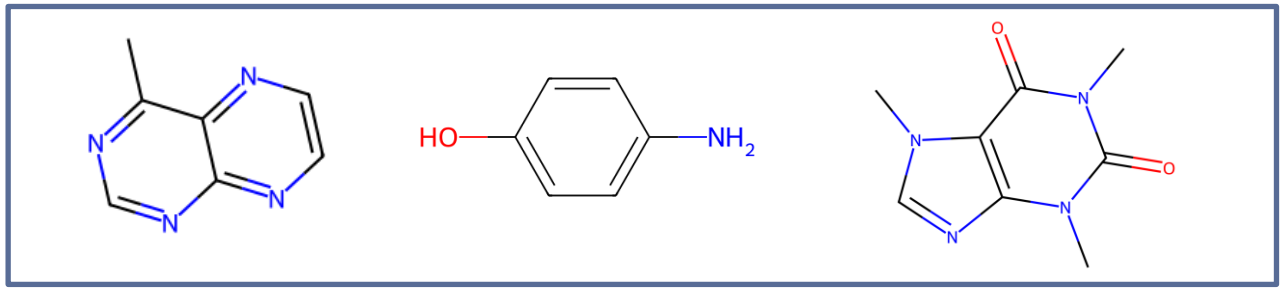
© Xuhui Huang

k-Nearest Neighbor (kNN)

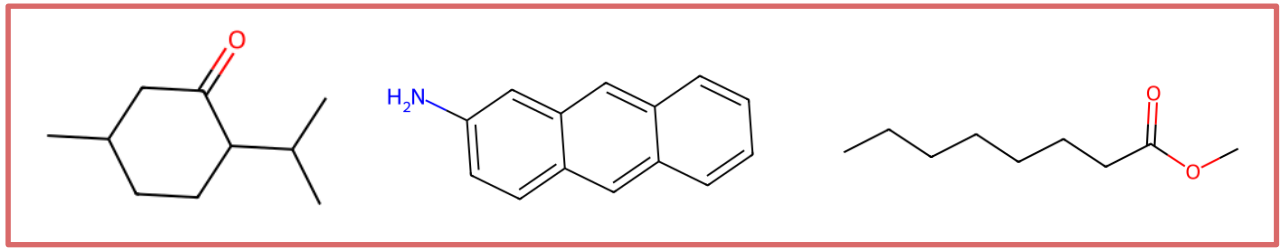
kNN assigns the same label (y) to "nearby" feature vectors (x). As a non-parametric supervised ML method, it does not require fitting parameters.

k=1: 1-Nearest Neighbor (1NN)

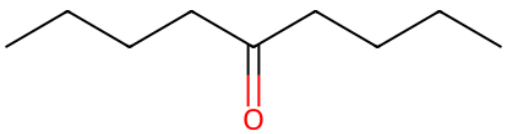
Training/learning: given $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$



Soluble



Non-soluble



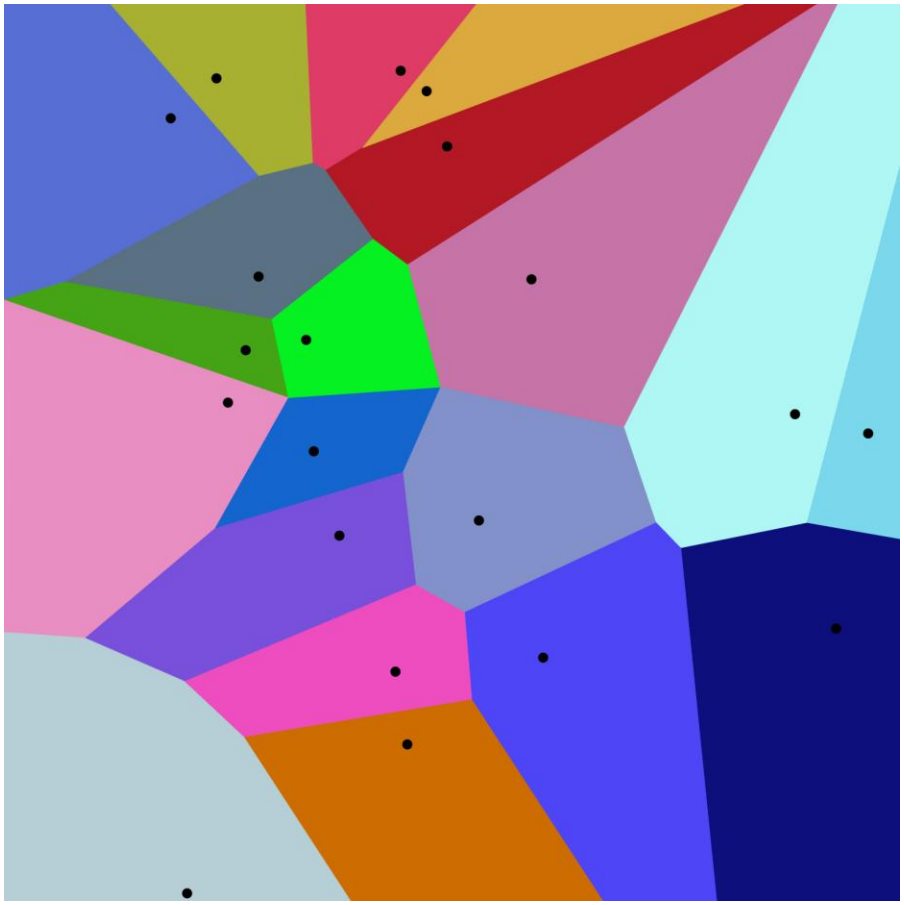
$\boldsymbol{x}$

Predication: For a new point x, find the nearest point $x^{(i)}$, return its label $y^{(i)}$.

© Xuhui Huang

1-Nearest Neighbor (1NN)

The decision regions of 1NN are defined by the “Voronoi Diagram”:



For a new point x, the nearest point $x^{(i)}$ is the seed point of the Voronoi cell that contains it.

Wikipedia

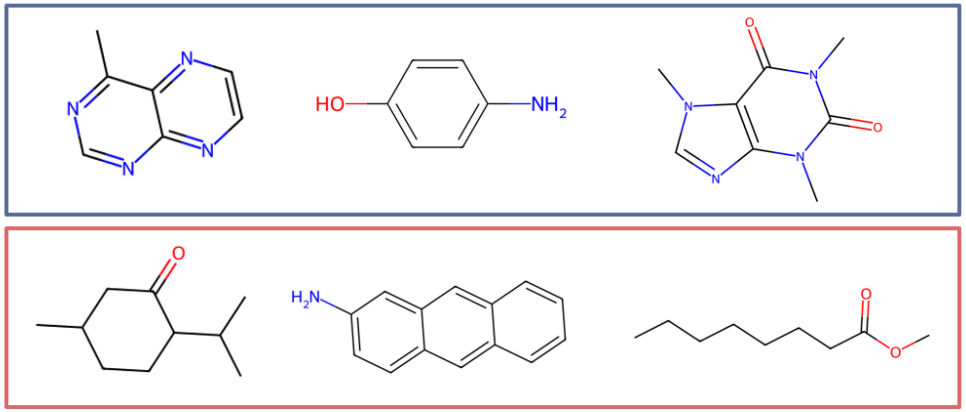
© Xuhui Huang

k-Nearest Neighbor (kNN)

Training/learning: given $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

Classification: For a new point x, find the k nearest training points $\{x^{(i)}\}_{i=1}^{k}$, and return the most popular class among the k points:

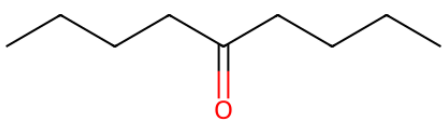
Given:



Soluble

Non-soluble

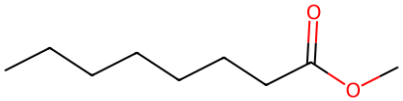
Prediction (3NN):



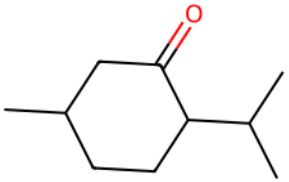
$\boldsymbol{x}$, non-soluble

The k=3 nearest points

Soluble



Non-soluble



Non-soluble

© Xuhui Huang

k-Nearest Neighbor (kNN)

Training/learning: given $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

Classification: For a new point x, find the k nearest training points $\{x^{(i)}\}_{i=1}^{k}$, and return the most popular class among the k points:

$$\hat{y} = arg \max_{y \in Y} \sum_{i=1}^{k} 1(y = y^{(i)})$$

Regression: For a new point x, find the k nearest training points $\{x^{(i)}\}_{i=1}^{k}$, and return the mean value of these k points:

$$\hat{y} = \frac{1}{k} \sum_{i=1}^{k} y^{(i)}$$

© Xuhui Huang

k-Nearest Neighbor (kNN)

How to define "nearest"? – different distance/similarity measures can be used:

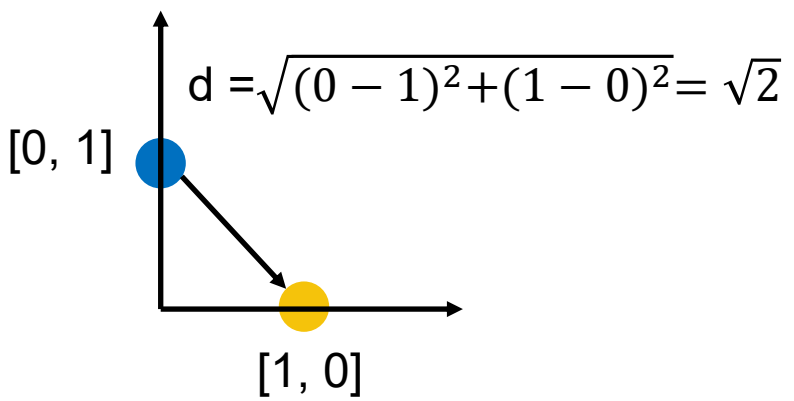
Discrete features: Hamming distance

$$d_H\left(x^{(i)}, x^{(j)}\right) = \sum_{a=1}^{d} 1\left\{x_a^{(i)} \neq x_a^{(j)}\right\}$$

Continuous features:

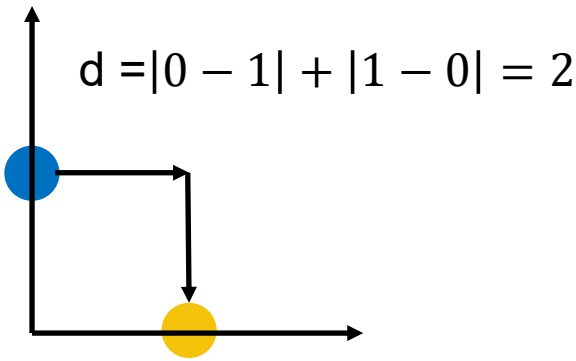
Euclidean distance

$$d\left(x^{(i)}, x^{(j)}\right) = \left(\sum_{a=1}^{d}\left(x_a^{(i)} - x_a^{(j)}\right)^2\right)^{\frac{1}{2}}$$



Manhattan distance

$$d\left(x^{(i)}, x^{(j)}\right) = \sum_{a=1}^{d}\left|x_a^{(i)} - x_a^{(j)}\right|$$



Standardization is recommended to normalize features!

© Xuhui Huang

k-Nearest Neighbor (kNN)

Standardization typically applied to normalize features – Do it for each input feature.

- Calculate the empirical mean and standard deviation for a feature in the training set:

Mean: $$\mu_\alpha = \frac{1}{n}\sum_{i=1}^{n} x_a^{(i)}$$

Standard deviation: $$\sigma_a = \left(\frac{1}{n}\sum_{i=1}^{n}\left(x_a^{(i)} - \mu_i\right)^2\right)^{\frac{1}{2}}$$

- Normalize features on both training and test points:

$$\tilde{x}_a^{(j)} = \frac{x_a^{(j)} - \mu_\alpha}{\sigma_a}$$

Prevents features with high magnitude or variance from dominating distance calculations.

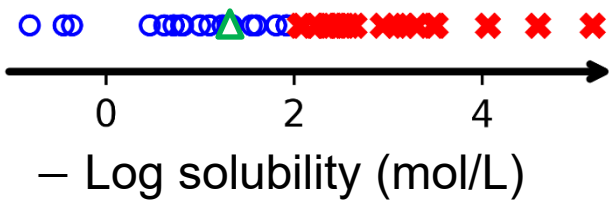
© Xuhui Huang

Exercise

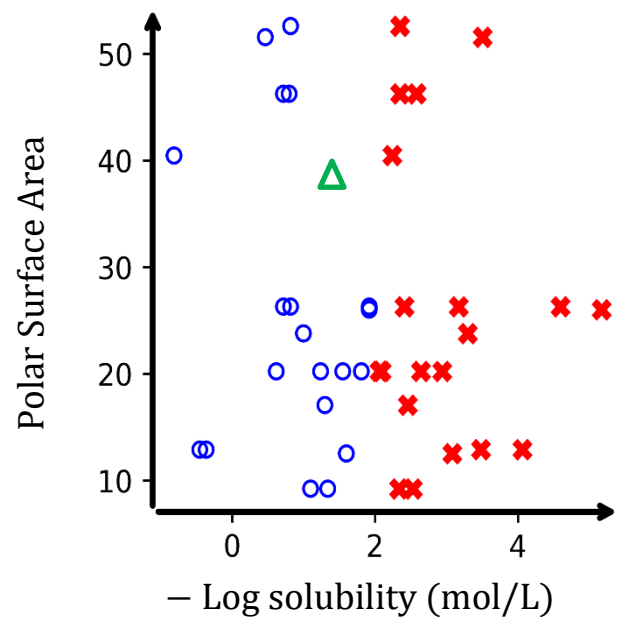
A training dataset consists of chemical compounds labelled as either **soluble** (-Log solubility ≤ 2, in blue circles) or **non-soluble** (-Log solubility > 2, in red crosses). A new compound, shown as a **green triangle**, needs to be classified using a **1-Nearest Neighbor (1NN) model**.

What label will the 1NN model assign to the green triangle when the Euclidean distance is computed using

(a). 1 feature?



(b). 2 features?



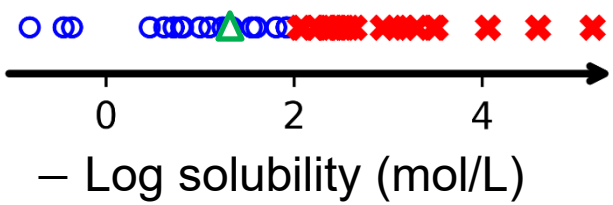
© Xuhui Huang

Exercise

A training dataset consists of chemical compounds labelled as either **soluble** (-Log solubility ≤ 2, in blue circles) or **non-soluble** (-Log solubility > 2, in red crosses). A new compound, shown as a **green triangle**, needs to be classified using a **1-Nearest Neighbor (1NN) model**.

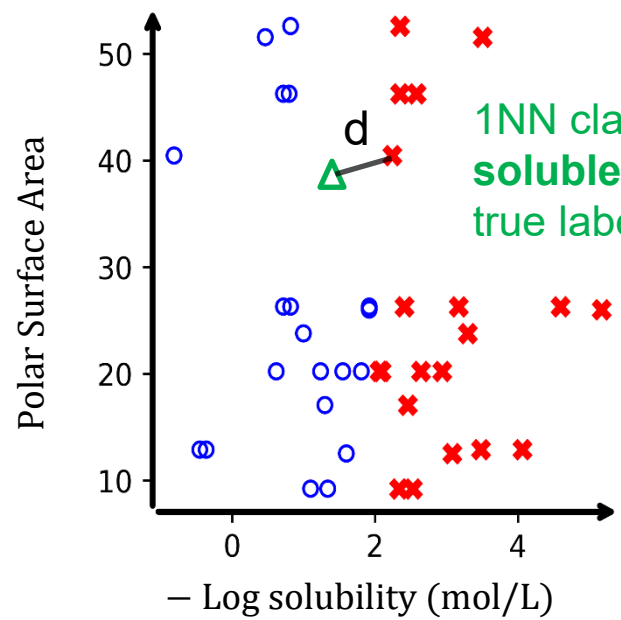
What label will the 1NN model assign to the green triangle when the Euclidean distance is computed using

(a). 1 feature?



1NN correctly classifies it as **"soluble".**

(b). 2 features?



1NN classifies it as **"non-soluble"**, even though its true label is "soluble".

© Xuhui Huang

k-Nearest Neighbor (kNN)

Pros:

- Easy to understand and implement
- Non-parametric – no training
- Can be used for both classification and regression

Cons:

- Performance can be affected by irrelevant or redundant (correlated) features.
- Prediction stage can be expensive due to distance calculations.

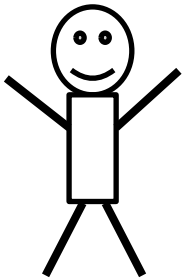
© Xuhui Huang

Outline

- Parametric Methods
  - Linear Regression
  - Logistic Regression
  - Avoiding Overfitting
- Non-parametric Methods
  - K-Nearest Neighbors (KNN)
  - **Decision Trees**
  - Random Forest

© Xuhui Huang

Decision Tree



Decision: Go outside or not

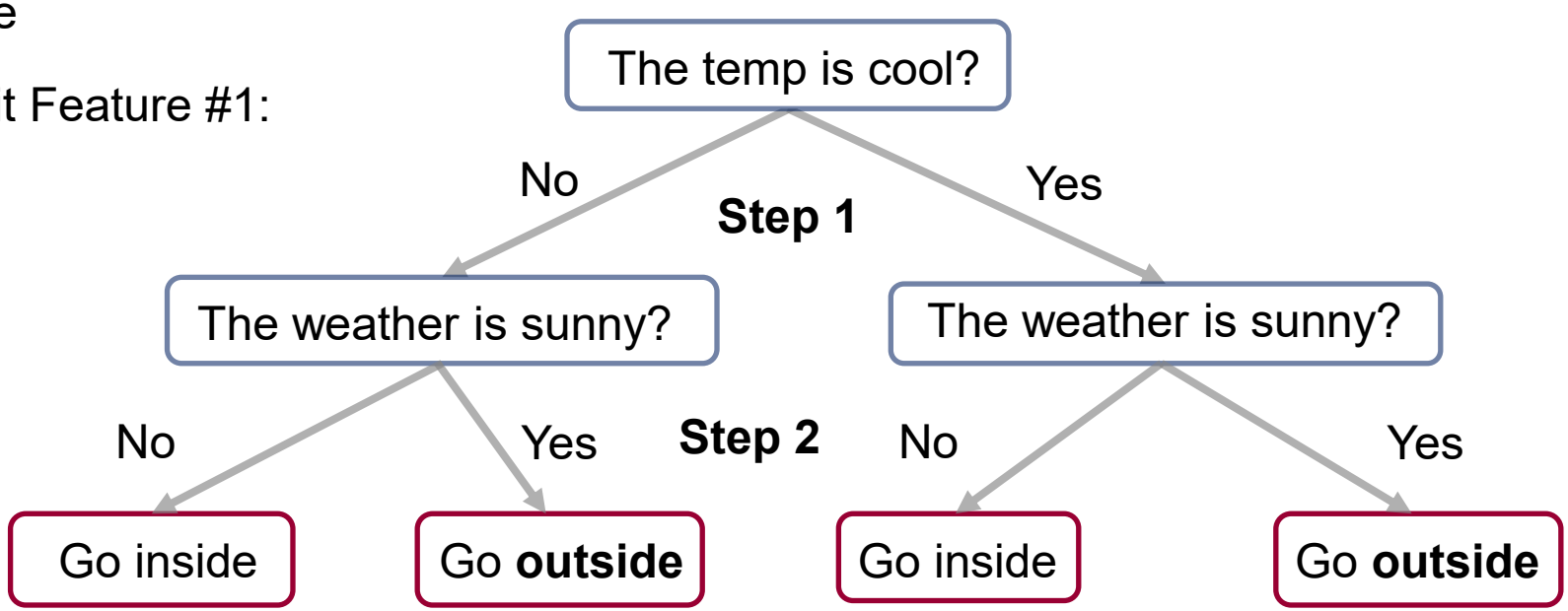
- Each node splits/tests 1 feature
- Each branch represents one outcome of the split/test

Weather (Feature #1)	Temperature (Feature #2)	Go outside
Sunny	Hot	Yes
Rainy	Hot	No
Sunny	Cool	Yes
Rainy	Cool	No

Step 1: Split feature #2: Temperature

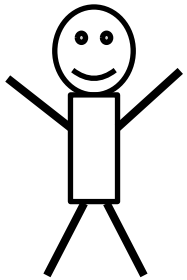
Step 2: Split Feature #1: Weather

Decision Tree #1 (2-layer)



© Xuhui Huang

Decision Tree



Decision tree:

- Each node splits/tests 1 feature
- Each branch represents one outcome of the split/test

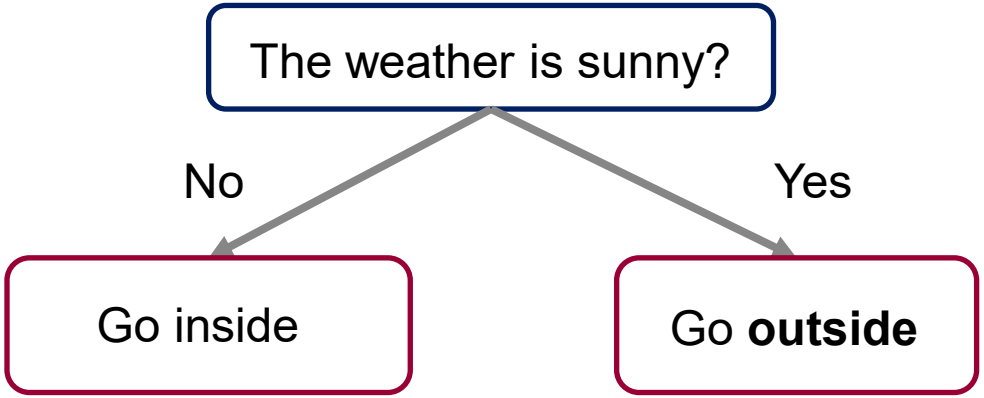
Weather (Feature #1)	Temperature (Feature #2)	Go outside
Sunny	Hot	Yes
Rainy	Hot	No
Sunny	Cool	Yes
Rainy	Cool	No

Step 1: Split feature #1: Weather

Step 2: Split feature #2: Temperature

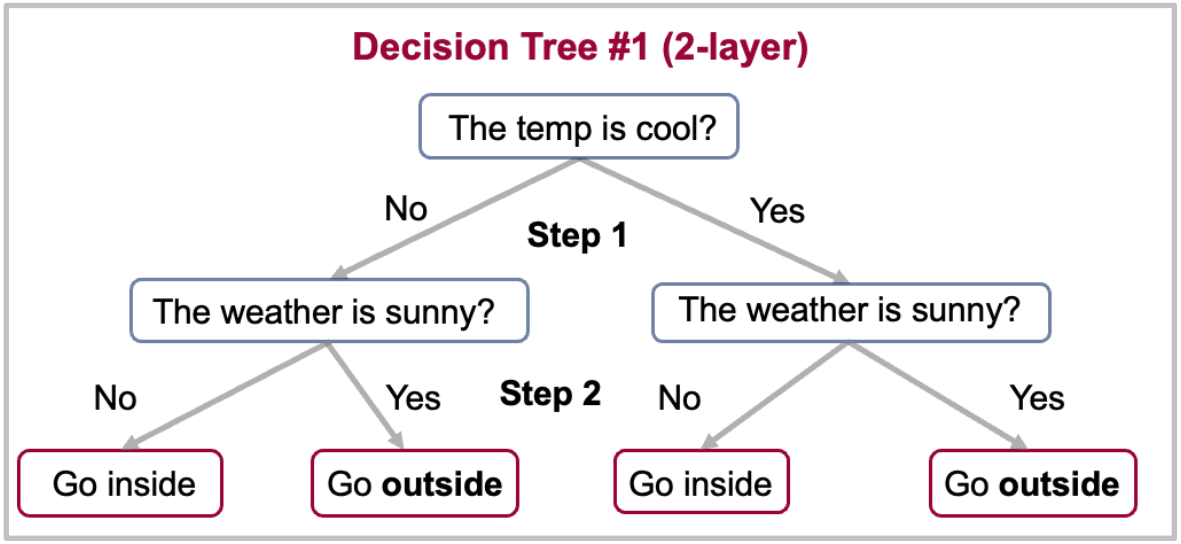
- Step 2 is unnecessary since the decision can be made after Step 1!

Decision Tree #2 (1-layer)

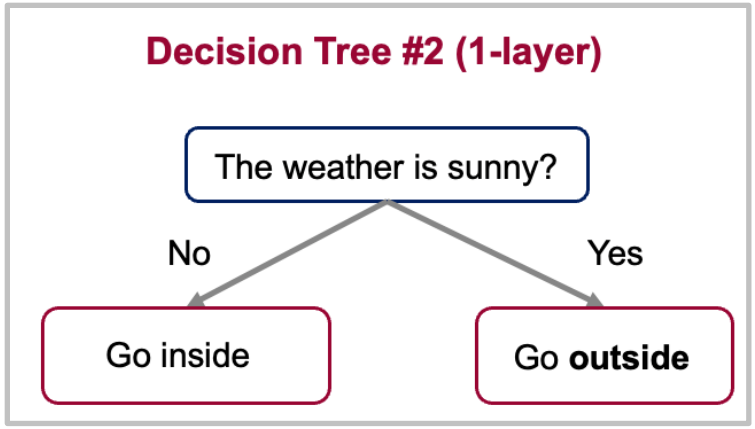


© Xuhui Huang

Decision Tree

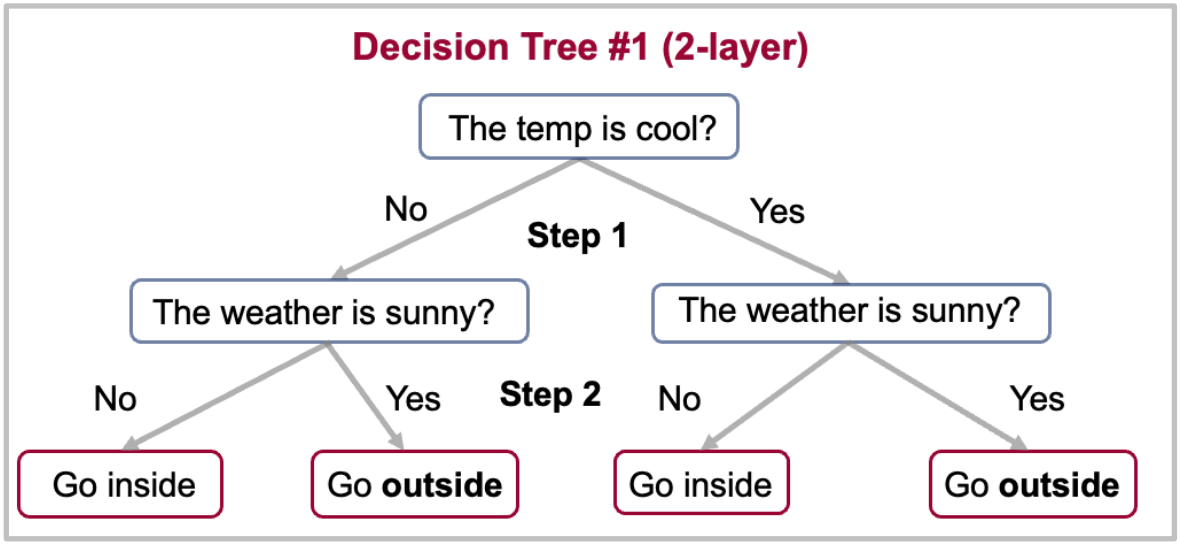


Both Tree #1 and Tree #2 can make correct decisions. Which one is better?



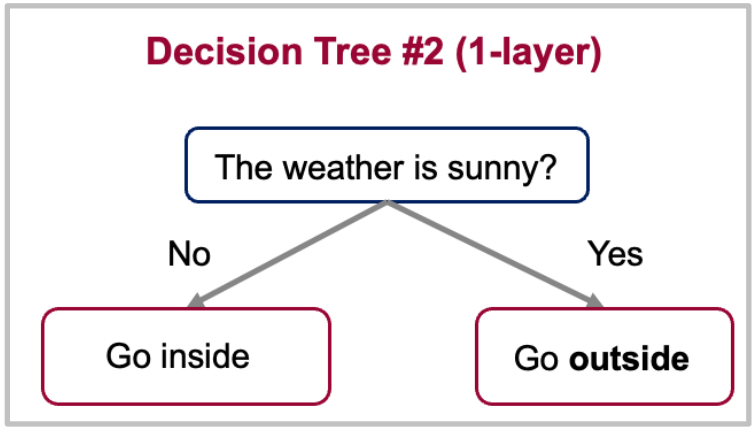
© Xuhui Huang

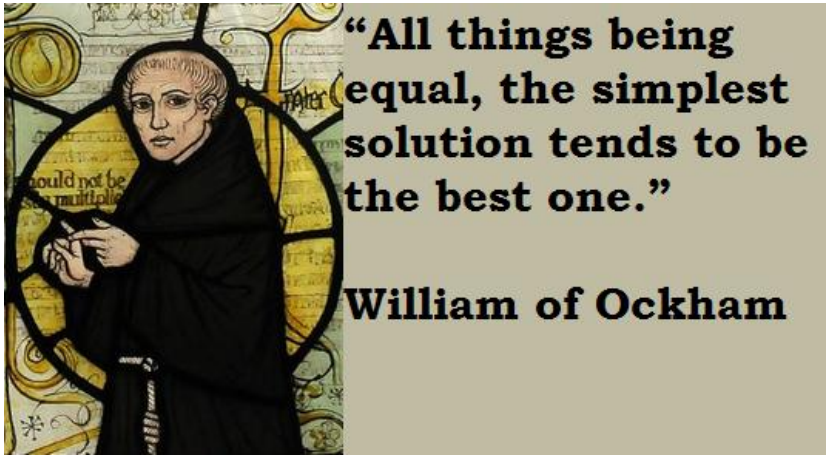
Decision Tree



Both Tree #1 and Tree #2 can make correct decisions. Which one is better?

Tree #2 is better!





Occam's razor

© Xuhui Huang

Decision Tree: Finding Optimal Splits

How do we select the best feature to split on at each step?

- **The simplest decision tree** that accurately classifies the training instances is more likely to generalize well.

Can we analytically determine the smallest possible decision tree that perfectly classifies the training set?

- No, this is not feasible!

We use an information-theoretic heuristic—**maximizing information gain**—to greedily select the best splits.

© Xuhui Huang

Decision Tree: Finding Optimal Splits

How to estimate information gain: **InfoGain**?

- In information theory, reducing entropy (a measure of uncertainty) increases information gain.

Information entropy: $$H(y) = -\sum_{y \in \mathcal{Y}} p(y) \log_2 p(y)$$

Statistical Thermodynamics (Chem562: Physical Chemistry):

Entropy in Chemistry: $$S = -k \sum_i P(E_i) \ln P(E_i)$$

$$P(E_i) = \frac{n_i^*}{\Pi} = \frac{e^{-\beta E_i}}{\sum_i e^{-\beta E_i}} \qquad \beta = \frac{1}{kT}$$

E_i: energy of state *I*

k: Boltzmann constant

© Xuhui Huang

Decision Tree: Finding Optimal Splits

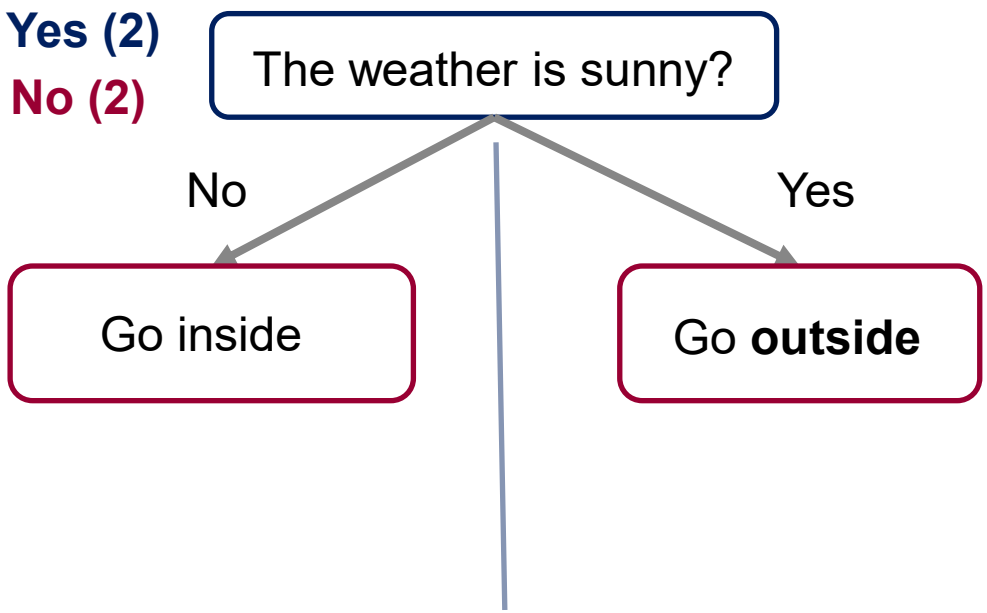
What is the information gain (entropy reduction) when splitting on **Feature #1 (Weather)?**

Weather (Feature #1)	Temperature (Feature #2)	Go outside
Sunny	Hot	Yes
Rainy	Hot	No
Sunny	Cool	Yes
Rainy	Cool	No

Information entropy:

$$H(y) = -\sum_{y \in \mathcal{Y}} p(y) \log_2 p(y)$$

- Before the split: $H_0(y) = -\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4} = 1$



© Xuhui Huang

Decision Tree: Finding Optimal Splits

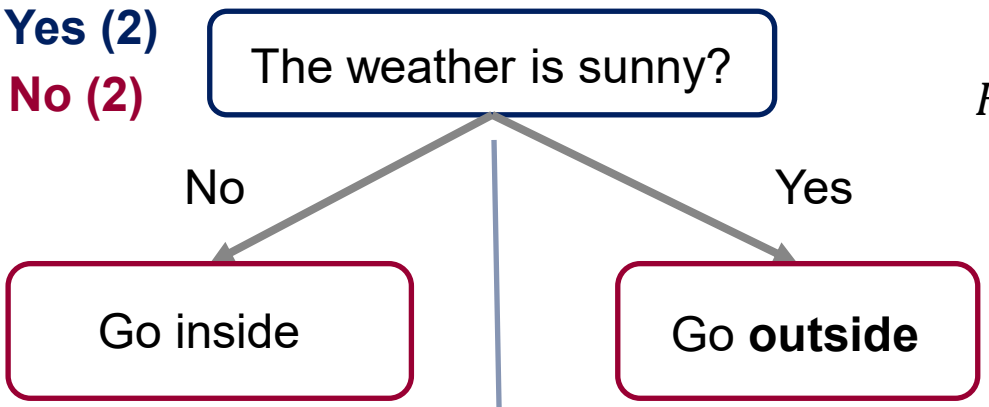
What is the information gain (entropy reduction) when splitting on **Feature #1 (Weather)?**

Information entropy: $$H(y) = -\sum_{y \in \mathcal{Y}} p(y) \log_2 p(y)$$

Weather (Feature #1)	Temperature (Feature #2)	Go outside
Sunny	Hot	Yes
Rainy	Hot	No
Sunny	Cool	Yes
Rainy	Cool	No

- Before the split: $H_0(y) = 1$

Yes (2)
No (2)



Subset 1: **No (2)**

$$H(y|\text{subset}_1) = -\frac{2}{2}\log_2\frac{2}{2} = 0$$

Subset 2: **Yes (2)**

$$H(y|\text{subset}_2) = -\frac{2}{2}\log_2\frac{2}{2} = 0$$

- After the split:

$$H_1(y) = \sum_{i=1}^{K} p(\text{subset}_\text{i})\, H(y|\text{subset}_\text{i})$$
$$= \frac{2}{4} \times 0 + \frac{2}{4} \times 0 = 0$$

- Information gain:

$$\textbf{InfoGain} = \Delta H$$
$$= H_0(y) - H_1(y) = \mathbf{1}$$

Split on Feature 1 has information gain!

© Xuhui Huang

Decision Tree: Finding Optimal Splits

What is the information gain (entropy reduction) when splitting on **Feature #2 (Temperature)?**

Weather (Feature #1)	Temperature (Feature #2)	Go outside
Sunny	Hot	Yes
Rainy	Hot	No
Sunny	Cool	Yes
Rainy	Cool	No

Information entropy: $$H(y) = -\sum_{y \in \mathcal{Y}} p(y) \log_2 p(y)$$

- Before the split: $H_0(y) = 1$

Yes (2)
No (2)

The temperature is cool?

No → Go inside

Yes → Go **outside**

Subset 1: **Yes (1)** **No (1)**

$$H(y|\text{subset}_1) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$$

Subset 2: **Yes (1)** **No (1)**

$$H(y|\text{subset}_2) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1$$

- After the split:

$$H_1(y) = \sum_{i=1}^{K} p(\text{subset}_\text{i})\, H(y|\text{subset}_\text{i})$$
$$= \frac{2}{4} \times 1 + \frac{2}{4} \times 1 = 1$$

InfoGain
$$= H_0(y) - H_1(y) = 0$$

No information gain! Split on Feature 2 is not optimal!

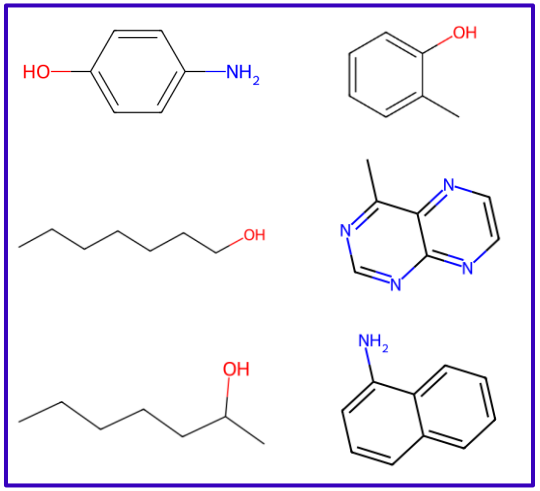
© Xuhui Huang

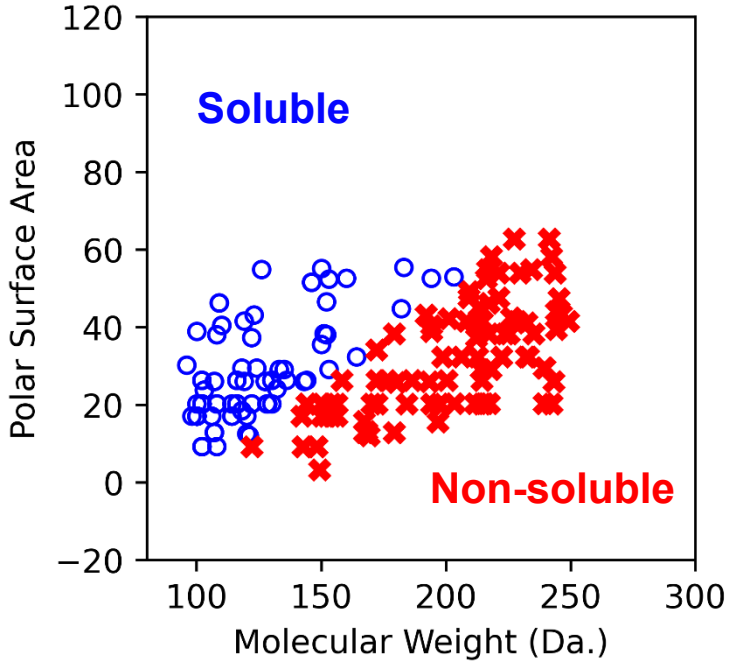
Decision Tree: Predicting Solubility

Task: Training a decision tree to predict **soluble** v.s. **non-soluble** compounds.

Soluble

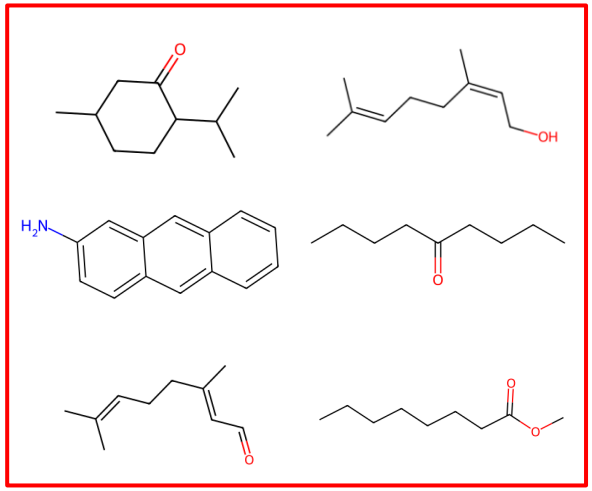
Log solubility ≥ -2





Non-soluble

Log solubility < -2



A subset of **200 compounds** from the Delaney Dataset.

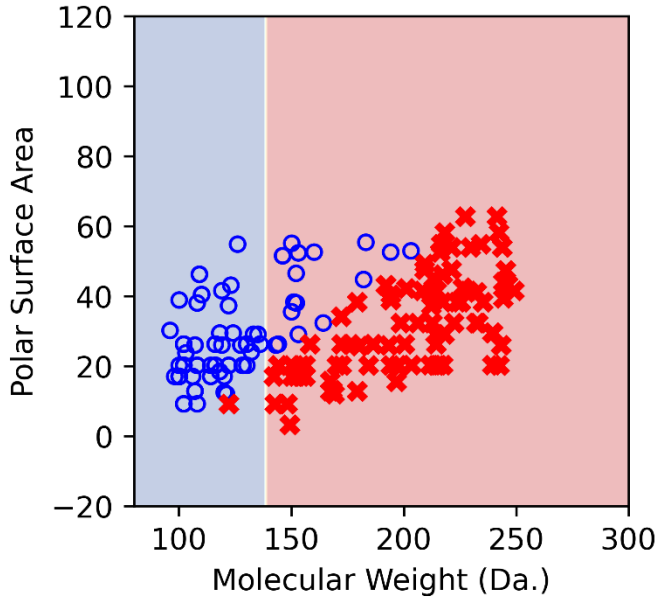
J. Chem. Inf. Comput. Sci. 44, 1000, (2004)

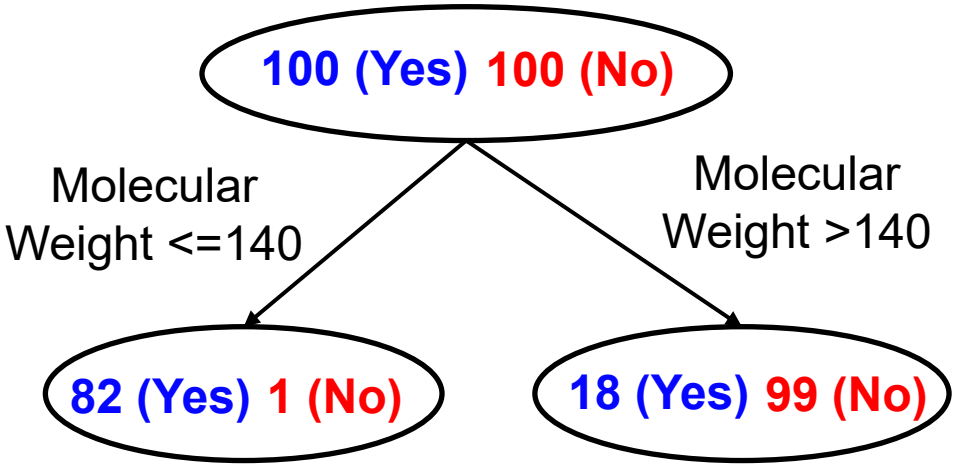
© Xuhui Huang

Decision Tree: Predicting Solubility

Example: Split on Molecular Weight with a threshold of 140.

$$\textbf{InfoGain} = H_0(y) - H_1(y)$$





$$\text{InfoGain} = H_0(y) - H_1(y|\text{Split}) = -\frac{100}{200}\log_2\frac{100}{200} - \frac{100}{200}\log_2\frac{100}{200} - H(y|\text{Split})$$

$$= 1 - \frac{83}{200}\left(-\frac{82}{83}\log_2\frac{82}{83} - \frac{1}{83}\log_2\frac{82}{83}\right) - \frac{117}{200}\left(-\frac{18}{117}\log_2\frac{18}{117} - \frac{99}{117}\log_2\frac{99}{117}\right)$$

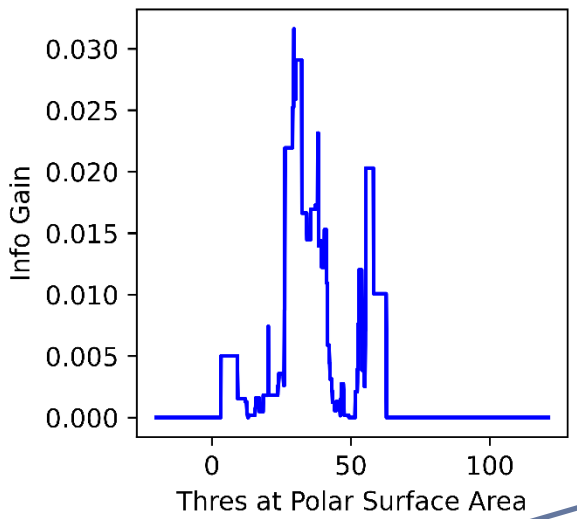
$$= 0.6$$

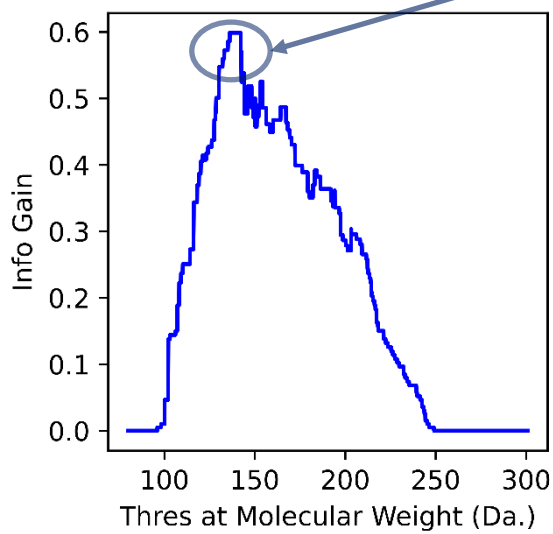
© Xuhui Huang

Decision Tree: Predicting Solubility

Scan both features (molecular weight & polar surface area) to find the optimal first split ("thresholds") that maximizes information gain.

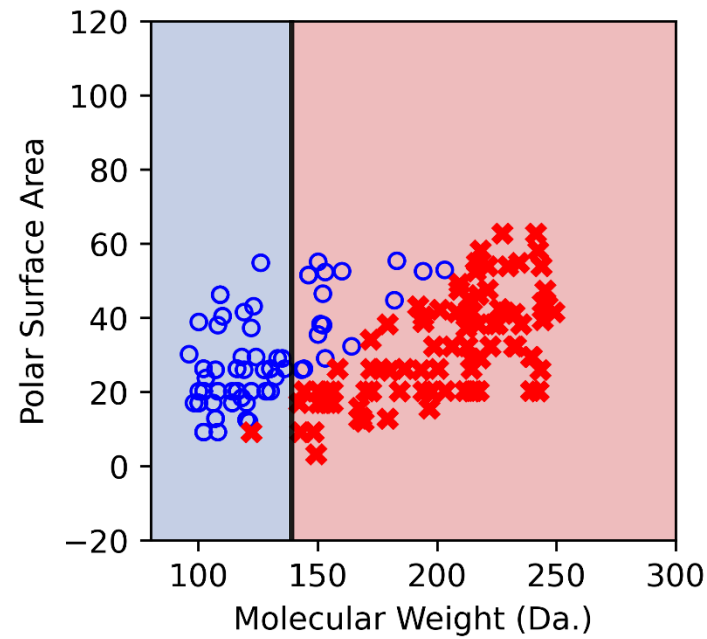
$$\textbf{InfoGain} = H_0(y) - H_1(y)$$





Maximum InfoGain occurs at MW=140

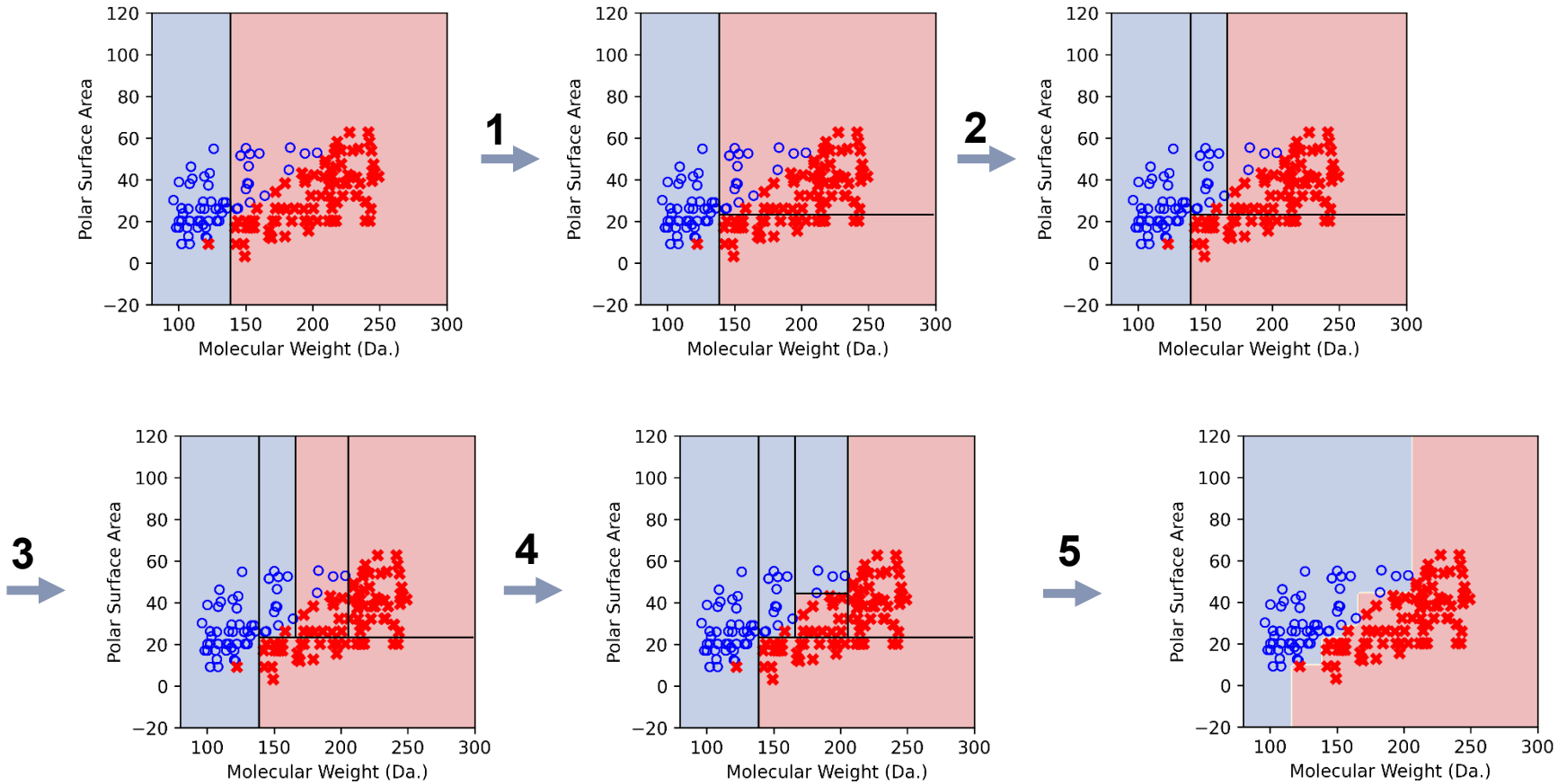
First split



© Xuhui Huang

Decision Tree: Predicting Solubility

Decision tree splits step-by-step:



Stop at **n=5** splits as all data in a subset belong to the same class!

© Xuhui Huang

Decision Tree: Stopping Criteria

Stopping Criteria for Decision Trees:

- Stop when a node contains a single data point.
- Stop when all instances in a subset belong to the same class.
- Stop when class impurity is below a predefined threshold.
- Stop when no further candidate splits remain.
- Stop when the tree reaches a predefined maximum depth.
- Stop when a node has fewer than the minimum required instances for a split.

© Xuhui Huang

Exercise

Consider a dataset containing 10 samples, where 50% of the samples have positive labels and 50% have negative labels. Each sample is represented by 3 numerical features. What is the maximum depth (# of layers) of a decision tree **trained** on this dataset?

(a) 2
(b) 9
(c) $\log_2(10)$
(d) $2^{10} - 1$

© Xuhui Huang

Exercise

Consider a dataset containing 10 samples, where 50% of the samples have positive labels and 50% have negative labels. Each sample is represented by 3 numerical features. What is the maximum depth (# of layers) of a decision tree **trained** on this dataset?

(a) 2
(b) 9
(c) $\log_2(10)$
(d) $2^{10} - 1$

- In the worst case, a decision tree keeps splitting until each sample is isolated in its own leaf node.
- Although there are only **3 features**, decision trees can repeatedly split on the same feature with different threshold values, so the number of features does not necessarily restrict depth.

© Xuhui Huang

Outline

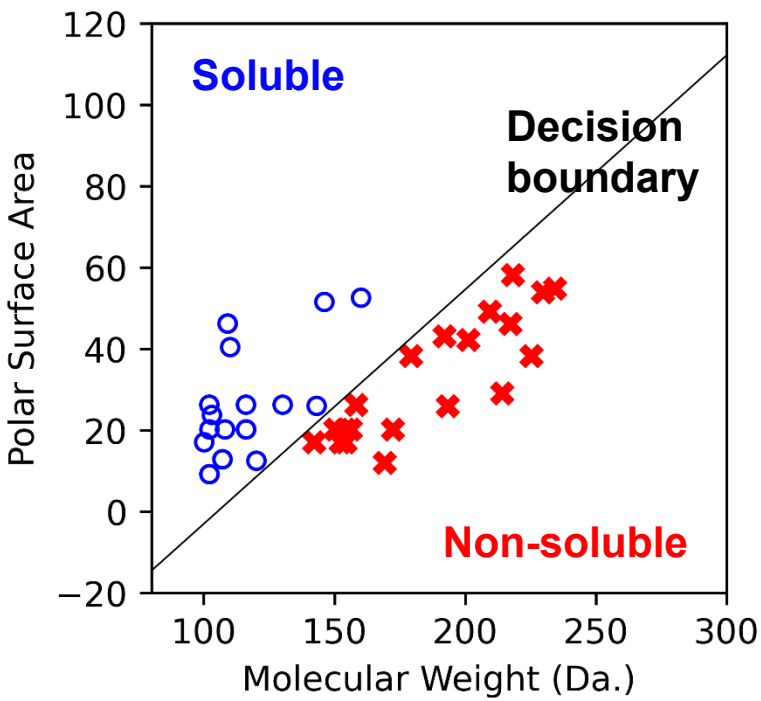
- Parametric Methods
  - Linear Regression
  - Logistic Regression
  - Avoiding Overfitting
- Non-parametric Methods
  - K-Nearest Neighbors (KNN)
  - Decision Trees
  - **Random Forest**

© Xuhui Huang

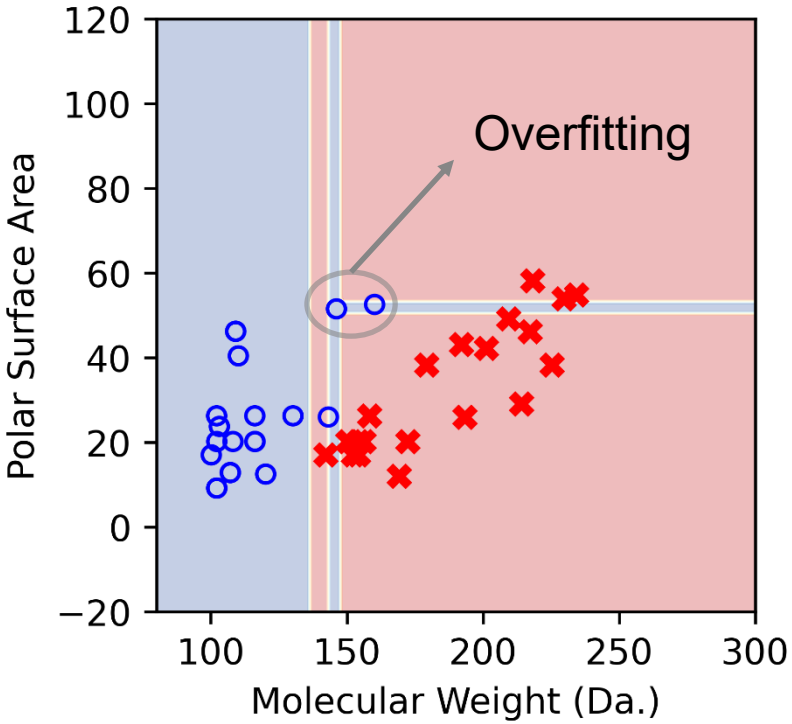
Random Forest

Decision trees are prone to overfitting, especially when they grow too deep, capturing noise and small fluctuations in the data.

Logistic regression



Decision tree (5-layers)



Poor generalization on unseen data!

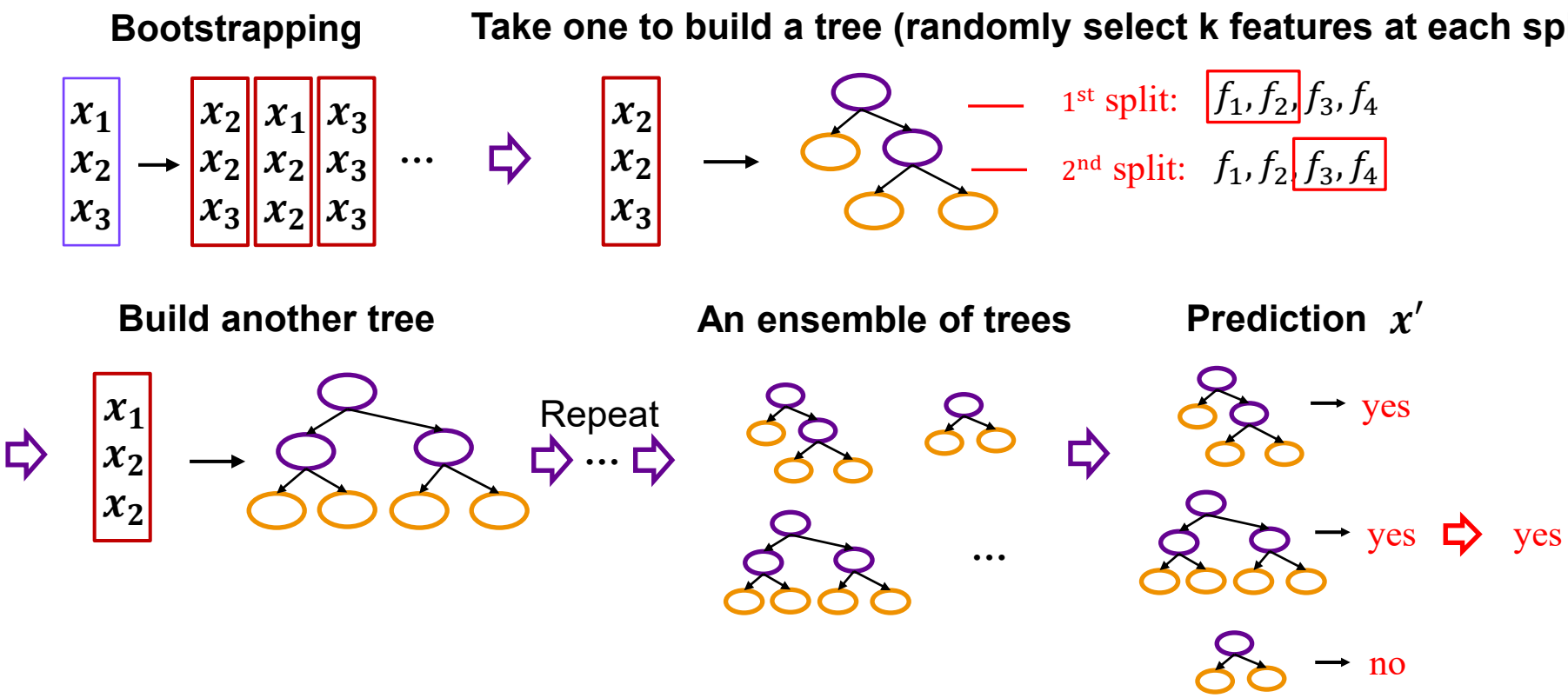
A subset of **40 compounds** from the Delaney Dataset.

J. Chem. Inf. Comput. Sci. 44, 1000, (2004)

© Xuhui Huang

Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees and combines their predictions—averaging for regression and majority voting for classification

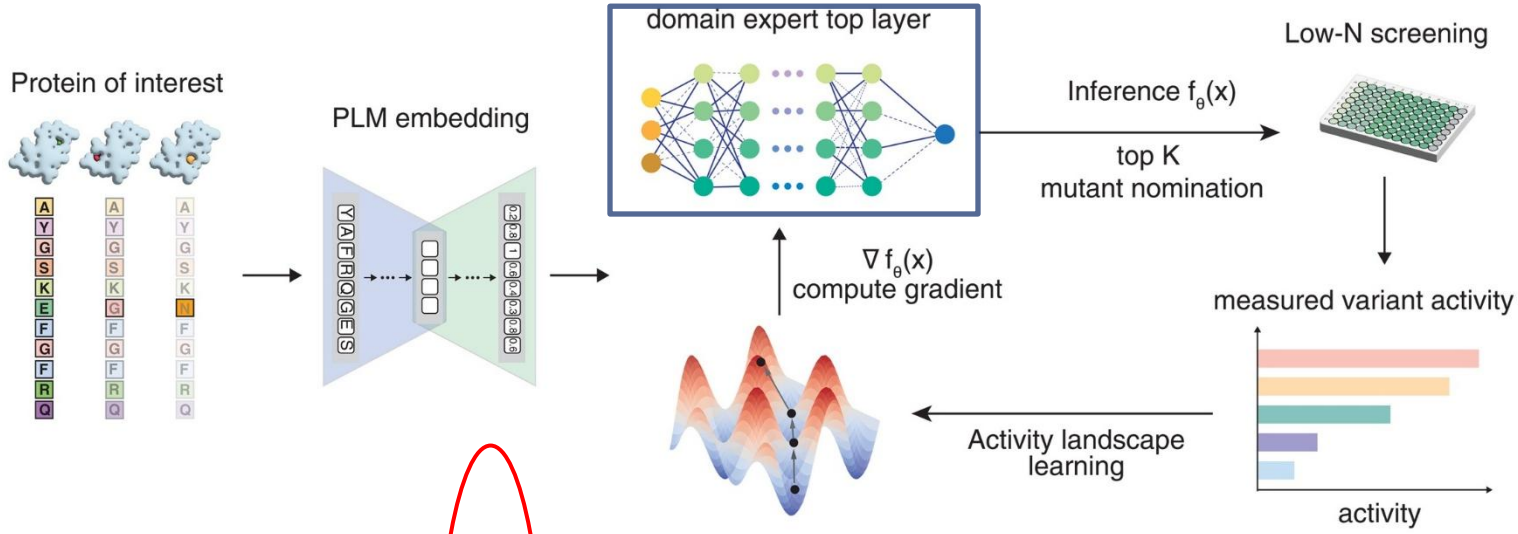


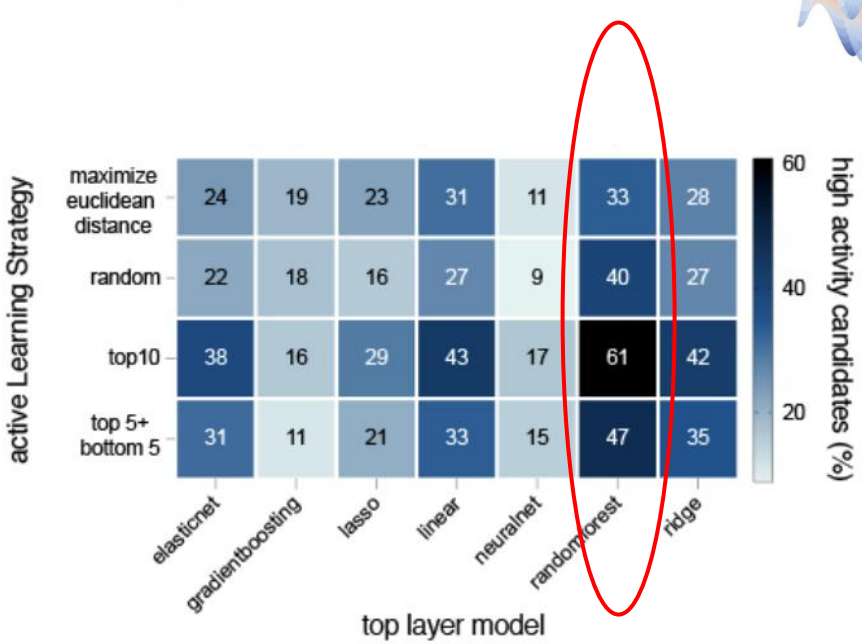
Breiman. *Machine Learning*, 45:5, (2001)

© Xuhui Huang

Random Forest

In silico directed evolution by a protein language model combined with random forest regression for designing protein functions.



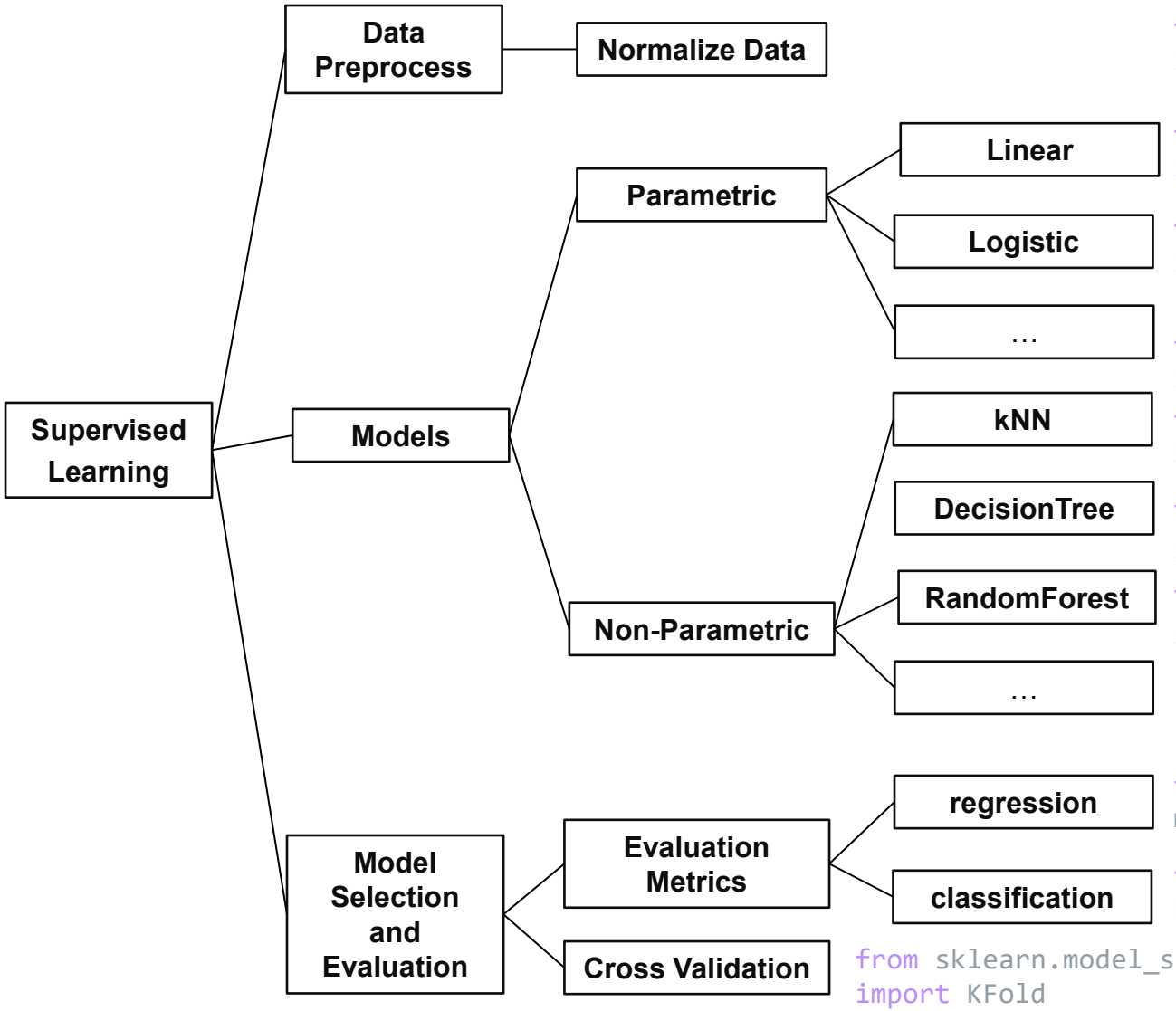


Random forest outperforms other approaches as a top layer regression model for the active learning to rapidly improve protein activity.

Science, 387,eadr6006, (2025)

© Xuhui Huang

Scikit Learn




```
from sklearn.preprocessing
import StandardScaler
```

```
from sklearn.linear_model
import LinearRegression
```

```
from sklearn.linear_model
import LogisticRegression
```

```
from sklearn.neighbors
import KNeighborsClassifier
```

```
from sklearn.neighbors
import KNeighborsRegressor
```

```
from sklearn.tree
import DecisionTreeClassifier
```

```
from sklearn.ensemble
import RandomForestClassifier
```

```
from sklearn.metrics import
mean_squared_error, r2_score
```

```
from sklearn.metrics import
roc_auc_score, confusion_matrix
```

```
from sklearn.model_selection
import KFold
```


© Xuhui Huang